

UNIwersytet Zielonogórski

Wydział Informatyki, Elektrotechniki i Automatyki

Praca dyplomowa

Kierunek: Computer Science

METODY REALIZACJI INTERFEJSU SIECIOWEGO W MODUŁACH IoT

Kacper Wojciechowski

Promotor:
dr inż. Mirosław Koziół

Pracę akceptuję:

.....

(data i podpis promotora)

Zielona Góra, styczeń 2022

Streszczenie

Niniejsza praca ma na celu dokonanie przeglądu dostępnych metod implementacji interfejsu sieciowego dla modułów IoT wykorzystujących mikrokontrolery serii Cortex-M. W trakcie realizacji pracy przedstawiono dziedzinę Internetu Rzeczy, uzasadniono konieczność wykorzystania przez wspomniane moduły interfejsów sieciowych, przybliżono czytelnikowi strukturę oraz protokoły składowe stosu TCP/IP, a także omówiono dostępne na rynku rozwiązania implementacyjne interfejsów sieciowych, takie jak moduły interfejsu Ethernet oraz interfejsu bezprzewodowego w paśmie 2,4 GHz. Ponadto, przedstawiona została przykładowa implementacja interfejsu sieciowego Ethernet oraz interfejsu bezprzewodowego w paśmie 2,4 GHz z wykorzystaniem platformy sprzętowej Nucleo firmy ST Microelectronics.

Słowa kluczowe: mikrokontroler, ST Microelectronics, interfejs sieciowy, Ethernet, interfejs bezprzewodowy.

Spis treści

1. Wstęp	1
1.1. Wprowadzenie	1
1.2. Cel i zakres pracy	1
1.3. Struktura pracy	2
2. Internet Rzeczy	3
2.1. Definicja Internetu Rzeczy	3
2.2. Dziedziny powiązane z konstrukcją i działaniem modułów IoT	3
2.3. Główne zalety oraz wady	4
2.4. Moduły IoT jako urządzenia sieci Internet	5
3. Stos TCP/IP	7
3.1. Charakterystyka stosu TCP/IP	7
3.2. Warstwa aplikacji	7
3.3. Warstwa transportowa	8
3.4. Warstwa sieciowa (Internetu)	9
3.5. Warstwa interfejsu sieciowego (dostępu do sieci)	11
3.5.1. Połączenie przewodowe	11
3.5.2. Połączenie bezprzewodowe	14
3.6. Implementacje stosu TCP/IP	15
4. Praktyczna implementacja interfejsu sieciowego	19
4.1. Założenia projektowe	19
4.2. Wybrane komponenty	19
4.2.1. Platforma sprzętowa	19
4.2.2. Środowisko i język programowania	19
4.2.3. Metody weryfikacji działania systemu	20
4.3. Implementacja części sprzętowej	20
4.3.1. Interfejs Ethernet	20
4.3.2. Interfejs bezprzewodowy	20
4.4. Implementacja części programowej	21
4.4.1. Interfejs Ethernet	21
4.4.2. Interfejs bezprzewodowy Wi-Fi	39
4.5. Testy rozwiązania	46
4.5.1. Interfejs Ethernet	46
4.5.2. Interfejs bezprzewodowy Wi-Fi	48
5. Dyskusja rezultatów i wnioski końcowe	51

Spis rysunków

2.1. Graficzne porównanie modeli ISO OSI oraz TCP/IP	6
3.1. Schemat transmisji z wykorzystaniem protokołu TCP	8
3.2. Schemat transmisji z wykorzystaniem protokołu UDP	9
3.3. Schemat drogi pakietu od nadawcy do odbiorcy w modelu OSI - Com- puter Notes	10
3.4. Numeracja wyprowadzeń gniazda 8P8C - Electronics Stack Exchange	11
3.5. Numeracja linii wtyku 8P8C - Electronics Stack Exchange	12
3.6. Schemat funkcjonalny sterownika Ethernet DB83848-P firmy Texas Instruments - Texas Instruments	12
3.7. Umieszczenie sterownika DB83848-P - Texas Instruments	12
3.8. Ułożenia przewodów w kablu UTP - The Internet Centre Inc.	13
3.9. Rodzaje anten dla urządzeń systemów wbudowanych - Circuit Digest	14
3.10. Shield Wi-Fi - Seeed Studio	14
3.11. Shield GSM - Botland	14
3.12. Moduł ENC28J60 - Aliexpress	15
3.13. Moduł W5500 - Aliexpress	15
3.14. Moduł ESP8266 - Nettigo	16
3.15. Moduł ESP32 - Botland	16
3.16. Platforma sprzętowa Nucleo-F429ZI posiadająca złącze Ethernet - - Botland	17
4.1. Złącze 8P8C obecne na płycie Nucleo-F429ZI	20
4.2. Schemat podłączenia modułu ESP8266	21
4.3. Wybór mikrokontrolera w STM32CubeIDE	21
4.4. Utworzenie projektu	22
4.5. Przełączenie zegaru HSE w tryb BYPASS	22
4.6. Konfiguracja sygnału zegarowego po przestawieniu HSE w tryb BY- PASS	22
4.7. Przypisanie trybu RMII do interfejsu Ethernet	23
4.8. Docelowe parametry interfejsu RMII	23
4.9. Schemat podłączenia interfejsu RMII	24
4.10. Uruchomienie stosu LwIP	25
4.11. Opcja LWIP_NETIF_LINK_CALLBACK	25
4.12. Konfiguracja zegarów	25
4.13. Konfiguracja interfejsów USART3 oraz USART6	39
4.14. Ustawienie zegara HSE	39
4.15. Częstotliwości końcowe w menadżerze zegarów	40
4.16. Lista skonfigurowanych linii GPIO	40
4.17. Podgląd parametrów linii PB3	41

4.18. Komunikacja za pomocą portu szeregowego	47
4.19. Sprawdzenie połączenia za pomocą polecenia ping	47
4.20. Komunikacja dwustronna z wykorzystaniem programu Hercules SE- TUP Utility	48
4.21. Zdjęcie przygotowanego układu	48
4.22. Komunikacja z komputerem nadzorującym cz.1	49
4.23. Komunikacja z kom-puterem nadzorującym cz. 2	49
4.24. Sprawdzenie połączenia za pomocą polecenia ping	49
4.25. Komunikacja dwustronna z wykorzystaniem programu Hercules SE- TUP Utility	50

Spis tabel

3.1. Spis linii interfejsu RMII	13
3.2. Zestawienie parametrów modułów Ethernet	16
3.3. Zestawienie parametrów modułów bezprzewodowych	17
4.1. Przyporządkowanie linii wyjściowych mikrokontrolera	24

Rozdział 1

Wstęp

1.1. Wprowadzenie

Wraz z rozwojem technologii, na rynku pojawia się coraz więcej innowacyjnych wynalazków wykorzystujących dotychczasowe osiągnięcia techniki informacyjnej i przyczyniających się do dalszego postępu technologicznego. Bardzo wyraźnym przykładem wspomnianego zjawiska jest przemysł informatyczny. Z upływem lat coraz wyraźniej zaobserwować można obecność urządzeń elektronicznych i komputerowych w różnych dziedzinach codziennego życia, począwszy od edukacji, przez opiekę zdrowotną, aż po relaks i rozrywkę. Nieodzowną rolę odgrywa tu rozwój Internetu, pozwalający na połączenie urządzeń w ogromne sieci do przesyłu informacji.

Wiele dostępnych obecnie produktów na rynku konsumenckim stanowi wypadkową szerzącej się automatyzacji i coraz większych korzyści płynących z mediów takich jak Internet. Dobrze obrazuje to dziedzina tzw. Internetu Rzeczy (ang. *Internet of Things, IoT*), złożonego z wielu rozproszonych modułów sprzętowych podłączonych do wspólnej sieci komputerowej w celu stworzenia większego systemu.

1.2. Cel i zakres pracy

Celem pracy jest dokonanie przeglądu metod realizacji interfejsu sieciowego w modułach IoT oraz praktyczna implementacja (sprzętowa i programowa) wybranej metody w systemie bazującym na mikrokontrolerze z procesorem Cortex-M.

Zakres pracy obejmował:

- przegląd możliwych do realizacji metod implementacji interfejsu sieciowego w modułach IoT bazujących na mikrokontrolerach z procesorami Cortex-M z uwzględnieniem m.in. wad i zalet każdej z nich,
- praktyczna realizacja modułu IoT w oparciu o mikrokontroler z procesorem Cortex-M z interfejsem sieciowym zrealizowanym według jednej z wybranych metod.

1.3. Struktura pracy

Rozdział pierwszy pracy zawiera wstęp oraz cel i zakres, w ramach którego zrealizowana została niniejsza praca dyplomowa, a także skróconą informację o poszczególnych sekcjach pracy.

Celem drugiego rozdziału jest zapoznanie Czytelnika z pojęciem Internetu Rzeczy (IoT) wraz z jego formalną definicją, pokazując zalety oraz wady tej dziedziny. Rozdział zwieńczony został uzasadnieniem interpretacji modułów IoT jako urządzeń sieciowych.

Trzeci z rozdziałów omawia stos TCP/IP oraz jego relację w stosunku do modelu ISO OSI. Jego treść przedstawia i krótko opisuje protokoły dostępne w poszczególnych warstwach modelu TCP/IP. Ponadto, przedstawia on także różnicę pomiędzy transmisją TCP a transmisją UDP, oraz omawia sposób podłączenia warstwy fizycznej.

Tematyką czwartego rozdziału jest praktyczna implementacja interfejsu Ethernet oraz interfejsu bezprzewodowego w paśmie 2,4 GHz dla platformy sprzętowej Nucleo-F429ZI. Rozdział przedstawia krok po kroku sposób konfiguracji urządzenia oraz przygotowany kod źródłowy do obsługi interfejsu.

Ostatni rozdział omawia rezultaty pracy oraz przedstawia wnioski wyciągnięte w trakcie jej realizacji.

Rozdział 2

Internet Rzeczy

2.1. Definicja Internetu Rzeczy

Zgodnie z definicją wskazaną w rekomendacji ITU-T Y.2060, Internet Rzeczy został określony jako „globalna infrastruktura dla społeczeństwa, umożliwiająca działanie zaawansowanym serwisom poprzez połączenie (fizyczne i wirtualne) przedmiotów bazujących na istniejących i ewoluujących interoperacyjnych technologiach komunikacji i wymiany informacji”[1].

W praktycznym aspekcie, powyższa definicja określa Internet Rzeczy jako złożone systemy udostępniające zaawansowane usługi poprzez komunikację i współpracę różnych urządzeń z wykorzystaniem rozmaitych mediów transmisyjnych, jak np. Internet.

2.2. Dziedziny powiązane z konstrukcją i działaniem modułów IoT

Moduły Internetu Rzeczy stanowią produkt działań specjalistów z wielu branż. Pomijając gałęzie nauki i przemysłu związane bezpośrednio z przeznaczoną funkcjonalnością konkretnego systemu, wyodrębnić można następujące dziedziny, których obecność można uniwersalnie zaobserwować w trakcie analizy różnych systemów.

- **Systemy wbudowane** – leżą one u podstaw oferowanych przez moduły IoT funkcjonalności. Każde z urządzeń wchodzące w skład omawianych sieci musi być wyposażone w układy cyfrowe umożliwiające pełnienie postawionych jemu zadań, takich jak np. dokonywanie pomiarów lub monitorowanie parametrów pracy innego urządzenia. Do ich realizacji wykorzystywane są mikrokontrolery sterujące urządzeniami peryferyjnymi (takimi jak czujniki, wyświetlacze, itp.), wykonujące wstępne przetwarzanie danych (ang. *preprocessing*), oraz umożliwiające komunikację z urządzeniem za pośrednictwem interfejsów komunikacyjnych (wliczając w to interfejsy szeregowo – np. SPI, I²C – jak i interfejsy sieciowe).

- **Analiza danych** – jednym z głównych zadań modułów sieci IoT jest zbieranie i generowanie danych, które następnie muszą zostać przetworzone przez jednostki o większej mocy obliczeniowej. W zależności od ilości danych oraz wymagań czasowych co do ich prędkości przetwarzania, krok ten jest realizowany na różnych poziomach hierarchii systemu. W celu filtracji, zorganizowania oraz wnioskowania na podstawie danych szeroko używane są formuły matematyczne i algorytmy, które pozwalają m.in. na zebranie powiązanych ze sobą informacji oraz przedstawienie ich w sposób umożliwiający wyciągnięcie głębszej konkluzji.
- **Sztuczna inteligencja** – twór bazujący na matematycznych zasadach analizy danych mający na celu znaczne przyspieszenie i automatyzację przetwarzania danych w sposób naśladujący biologiczną inteligencję. Pozwala ona na interpretację ogromnych ilości informacji, w czasie znacznie przekraczającym możliwości całych zespołów ludzi. Przekłada się to na znaczne poprawienie sprawności systemów IoT. Ponadto, sztuczna inteligencja niejednokrotnie stanowi integralną część funkcjonalności dostępnych dla użytkowników, szczególnie w systemach inteligentnych takich jak systemy *smart home* czy systemy zabezpieczeń (np. poprzez możliwość identyfikacji osoby na podstawie danych biometrycznych lub wyglądu).
- **Sieci komputerowe** – ze względu na konieczność wymiany danych pomiędzy urządzeniami leżącą u podstaw koncepcji Internetu Rzeczy, każdy moduł musi być wyposażony w interfejs komunikacyjny lub ich zespół w celu umożliwienia transferu danych zarówno z, jak i do urządzenia. Wspomniane wcześniej interfejsy składają się z interfejsu przewodowego Ethernet albo interfejsów bezprzewodowych WiFi lub GSM. Wymaga to zapewnienia bezpieczeństwa poprzez stosowanie techniki szyfrowania.
- **Druk 3D** – moduły IoT często pełnią innowacyjne i niestandardowe zadania, co ogranicza możliwości korzystania z dostępnych na rynku elementów, jak np. obudowy, elementy mocujące oraz elementy wykonawcze ruchomych mechanizmów. Ogromną przewagą odznacza się tutaj branża druku 3D pozwalająca na bardzo szybkie i łatwe przygotowanie elementów prototypowych oraz elementów dla finalnego produktu znacząco zmniejszając koszty produkcji.

2.3. Główne zalety oraz wady

Systemy Internetu Rzeczy są innowacyjną i bardzo elastyczną techniką, dzięki czemu charakteryzują się szeregiem zalet, spośród których najważniejszymi są:

- stanowienie przestrzeni do innowacyjnych pomysłów wpływających zarówno na standard życia społeczeństwa, jak i wydajność przemysłu oraz badań naukowych;
- zapewnianie większego bezpieczeństwa obiektów oraz personelu, co jest szczególnie istotne w miejscach pracy o podwyższonym ryzyku;
- umożliwienie monitorowania zmian zachodzących w otoczeniu pod wpływem różnych czynników i upływu czasu;

- gromadzenie wartościowych danych pomiarowych, które są zbyt obszerne lub niemożliwe do wykrycia bezpośrednio przez człowieka (np. ustalenie konkretnego poziomu zanieczyszczenia powietrza).

Duża część zaprezentowanych zalet wynika z metodyki konstruowania modułów i projektowania systemów Internetu Rzeczy wykorzystującej różnorodne elementy o szerokim wachlarzu zastosowań do konstrukcji układów przeznaczonych do ściślej określonych zadań. Jako przykład można wskazać dostępne systemy typu *smart home*, które przy użyciu elementów takich jak czujniki wilgoci, czujniki dymu i czujniki drgań są w stanie nadzorować bezpieczeństwo domu (zarówno chroniąc przed pożarem, jak i np. włamaniem).

Podobnie, jak w przypadku wszystkich technologii, systemy IoT nie mogą obejść się bez wad wynikających z obecnego stopnia zaawansowania technologicznego, jakości komponentów, czy też założeń projektowych tworzonych rozwiązań. Największe z nich to:

- mocno ograniczona moc obliczeniowa końcowych urządzeń pomiarowych, wymuszająca połączenie z centralnym punktem o znacznie większej mocy w celu przetworzenia dużych pakietów danych;
- wymóg zakupu zestawu urządzeń w celu utworzenia kompletnego, w pełni funkcjonalnego systemu, co zniechęca klientów indywidualnych;
- konieczność obsługi komunikacji z wieloma urządzeniami oddalonymi od siebie na zróżnicowane odległości (niekiedy bardzo duże, szczególnie w przypadku systemów o przeznaczeniu przemysłowym i środowiskowym).

Stosunek wad do zalet rozwiązań Internetu Rzeczy jest kwestią dyskusyjną i w pełni zależną od konkretnych przypadków zastosowań.

2.4. Moduły IoT jako urządzenia sieci Internet

Sieć Internet stanowi bardzo popularne medium transmisyjne w wielu domach, biurach oraz budynkach przemysłowych. Jej koncepcja – stanowiąca jedną z podstaw dziedziny Internetu Rzeczy – sprawiła, że większość modułów IoT obecnie wykorzystuje dostęp do Internetu jako fundament systemu, którego są częścią. Szczególnie zauważalne jest to w przypadku systemów przeznaczonych dla klientów indywidualnych oraz firm biurowych.

W celu komunikacji z wykorzystaniem Internetu, moduły IoT posiadają potrzebę implementacji stosu komunikacyjnego bazującego na modelu OSI (ang. *Open System Interconnection Reference Model*) autorstwa Międzynarodowej Organizacji Normalizacyjnej ISO (ang. *International Organization for Standardization*). Stanowi on 7-warstwową hierarchię elementów systemu komunikacyjnego z podziałem ze względu na pełnione role. Zgodnie z treścią prezentacji autorstwa pracownika Katedry Inteligentnych Systemów Informatycznych Politechniki Częstochowskiej profesora nadzwyczajnego Roberta Nowickiego [2] poszczególne warstwy modelu OSI (począwszy od najwyższej) pełnią następujące funkcje:

- 1) warstwa aplikacji – odpowiada za standardowe funkcjonalności programów, takie jak m.in. przeglądanie stron WWW, oraz zapewnia wspólne API dla różnorodnych usług;
- 2) warstwa prezentacji – pełni funkcję formatowania danych do ogólnie udostępnionej postaci wykorzystującej zdefiniowaną składnię;
- 3) warstwa sesji – powiadamia użytkownika o błędach występujących w niższych warstwach, zapewnia synchronizację i zarządzanie sesjami pomiędzy usługami;
- 4) warstwa transportowa – nadzoruje połączenia występujące w warstwie sieciowej, powiązane z nimi zdarzenia jak np. upływ czasu przetwarzania (ang. *timeout*) oraz prawidłowość przesyłu sekwencji pakietów;
- 5) warstwa sieciowa – pełni funkcje adresacji sieci, podziału i scalania pakietów na fragmenty, a także podejmuje decyzję o trasie którą pakiety zostaną przesłane;
- 6) warstwa łącza danych – zajmuje się tworzeniem ramki transmisyjnej, określaniem zasad transmisji i nadzorowaniem przebiegu wymiany danych w warstwie fizycznej;
- 7) warstwa fizyczna – pełni rolę fizycznego medium przesyłu danych.

Na podstawie powyższego modelu utworzony został stos protokołów komunikacyjnych TCP/IP, w którym warstwy aplikacji, prezentacji oraz sesji zostały nazwane zbiorczo warstwą aplikacji, a warstwy łącza danych i fizyczna zostały połączone w warstwę interfejsu sieciowego, tworząc hierarchię 4-warstwową. Na każdym poziomie wyróżnić można grupę protokołów odpowiedzialnych za przystosowanie nadawanych danych do transmisji, tzw. enkapsulację (ang. *encapsulation*), oraz przetworzenie otrzymanych danych z postaci ramki transmisyjnej na postać użyteczną dla usługi końcowej (np. przeglądarki internetowej) [3]. Graficzne porównanie warstw obu modeli przedstawione zostało na rysunku 2.1.

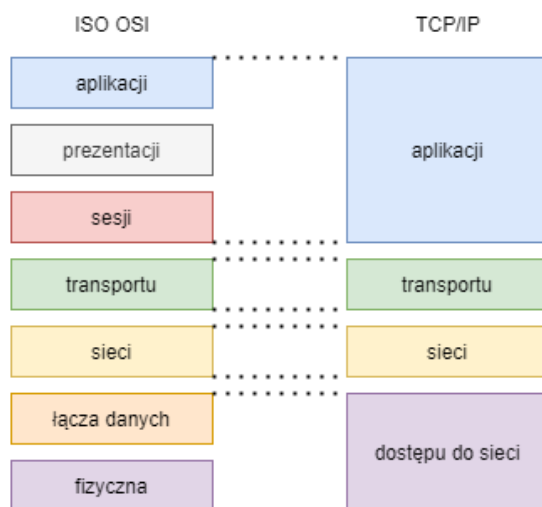


Figure 2.1. Graficzne porównanie modeli ISO OSI oraz TCP/IP

Rozdział 3

Stos TCP/IP

3.1. Charakterystyka stosu TCP/IP

Stos TCP/IP, jako uproszczona wersja modelu OSI, pozwala na prostą klasyfikację protokołów ze względu na pełnione przez nie funkcje oraz ich poziom abstrakcji. Niższe warstwy stosu skupiają się w szczególności na aspektach sprzętowych, natomiast wyższe warstwy przykładają większą uwagę do aspektów funkcjonalnych. Niniejszy rozdział omawia poszczególne warstwy stosu TCP/IP w kolejności od najwyższej do najniższej.

3.2. Warstwa aplikacji

Stanowi ona warstwę pośredniczącą pomiędzy programami komputerowymi, a pozostałymi warstwami stosu TCP/IP. W zależności od funkcjonalności udostępnianej przez program końcowy w warstwie aplikacji dostępne są protokoły o zróżnicowanych zastosowaniach [4].

- 1) HTTP, HTTPS (ang. *Hypertext Transfer Protocol*, *Hypertext Transfer Protocol Secure*) – protokoły odpowiedzialne za przesyłanie pomiędzy serwerem a urządzeniem końcowym stron WWW w postaci formularzy, np. HTML w odpowiednio nieszyfrowanej i szyfrowanej wersji [5];
- 2) FTP, TFTP, NFS (ang. *File Transfer Protocol*, *Trivial File Transfer Protocol*, *Network File System*) – protokoły transmisji plików pomiędzy dwoma urządzeniami za pośrednictwem sieci Internet;
- 3) SMTP, POP3, IMAP (ang. *Simple Mail Transfer Protocol*, *Post Office Protocol*, *Internet Message Access Protocol*) – zbiór protokołów odpowiedzialny za komunikację za pomocą wiadomości e-mail [6][7];
- 4) Telnet – protokół umożliwiający dostęp zdalny do innego komputera, pozwalając na logowanie oraz przesyłanie wprowadzanych znaków z komputera źródłowego do komputera docelowego [8];
- 5) SNMP (ang. *Simple Network Management Protocol*) – protokół do zarządzania urządzeniami podłączonymi do sieci komputerowej, wyposażonymi w agenta

SNMP. Pozwala on w sposób zdalny konfigurować urządzenia sieciowe takie jak np. routery, serwery lub przełączniki [9];

- 6) DNS (ang. *Domain Name System*) – protokół odpowiedzialny za translację nazw domenowych na adresy IP urządzeń sieciowych (serwerów) pełniących rolę hosta dla danej strony;
- 7) IRC (ang. *Internet Relay Chat*) – protokół w modelu klient-serwer umożliwiający komunikację na żywo za pomocą komunikatorów tekstowych (tzw. *chatów*) [10];
- 8) DHCP (ang. *Dynamic Host Configuration Protocol*) – odpowiada za dynamiczną konfigurację urządzeń końcowych podłączonych do sieci komputerowej poprzez nadanie im tzw. dynamicznego adresu IP z dostępnej puli adresowej.

Pomimo różnej konstrukcji informacji wytworzonej przez wymienione protokoły, w niższych warstwach są one uniwersalnie interpretowane jako sekcja danych, która następnie jest obudowywana przez protokoły warstw niższych w procesie tzw. enkapsulacji. Do komunikacji pomiędzy warstwą aplikacji a warstwą transportową wykorzystywane są tzw. gniazda sieciowe (ang. *Internet Sockets*) będące konstrukcjami programowymi zawierającymi informacje o adresie IP, numerze portu i wykorzystywanym protokole [4].

3.3. Warstwa transportowa

W warstwie transportowej otrzymane z warstwy aplikacji dane są dzielone na szereg pakietów i obudowywane nagłówkiem zawierającym m.in. takie informacje jak numer portu nadawcy i odbiorcy wiadomości, który jest istotny ze względu na rozpoznanie, do którego protokołu warstwy aplikacji odebrana wiadomość ma zostać przekazana. Wykorzystywane na tym poziomie protokoły określają cechy przebiegu transmisji. Głównymi protokołami tej warstwy są TCP i UDP.

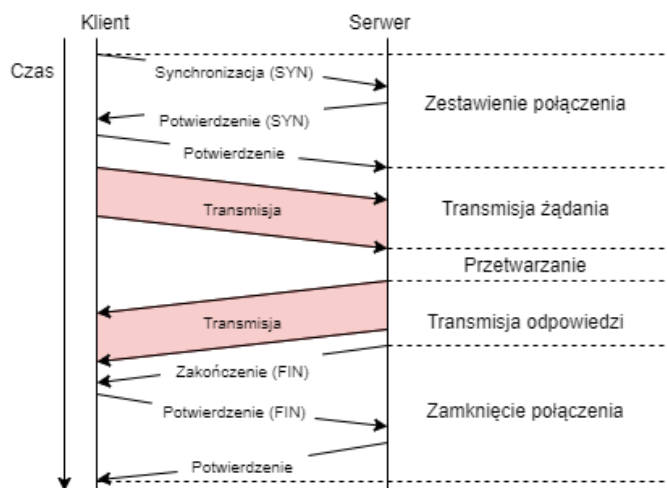


Figure 3.1. Schemat transmisji z wykorzystaniem protokołu TCP

TCP (ang. *Transmission Control Protocol*) jest protokołem połączeniowym (ang. *connection oriented*), co oznacza, że na początku transmisji tworzy on sesję pomiędzy nadawcą a odbiorcą. Ponadto cechuje się niezawodnością, poprzez wymaganie potwierdzenia otrzymania każdego przesłanego pakietu i wykonanie retransmisji w przypadku błędów transmisji lub braku potwierdzenia. Cecha ta jest szczególnie istotna w przypadku przesyłania danych, których integralność odgrywa kluczową rolę, np. plików lub wiadomości e-mail. Głównymi protokołami w warstwie aplikacji wykorzystującymi protokół TCP są: HTTP, HTTPS, FTP, SMTP i Telnet [4]. Przebieg pojedynczej transmisji został przedstawiony na rysunku 3.1.

Protokół UDP (ang. *User Datagram Protocol*), w przeciwieństwie do TCP, jest protokołem bezpołączeniowym i nie wykorzystuje retransmisji. Oznacza to, że nadanie wiadomości z wykorzystaniem protokołu UDP nie gwarantuje jej dotarcia do odbiorcy. Dodatkowo, kolejność otrzymywanych pakietów nie musi odpowiadać kolejności ich wysłania, co wymusza ich sortowanie przez urządzenie odbiorcy. Zaletą protokołu UDP jest natomiast większa szybkość transmisji dzięki znacznemu uproszczeniu mechanizmów w nim występujących. Szczególnie przydatne jest to w przypadku usług, gdzie zgubienie pojedynczych pakietów nie stanowi problemu, a ważniejsze jest zachowanie bardzo niskiego opóźnienia wynikającego z transmisji. Dotyczy to szczególnie komunikacji głosowej oraz gier komputerowych w trybie on-line. Przykładami protokołów wykorzystujących transmisję UDP są: DNS, DHCP, TFTP, SNMP, RIP, VOIP [4]. Przebieg pojedynczej transmisji został przedstawiony na rysunku 3.2.

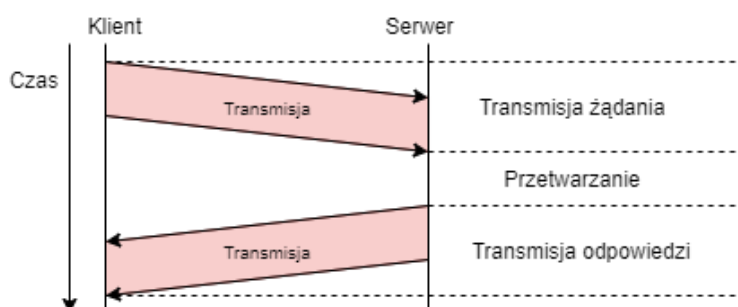


Figure 3.2. Schemat transmisji z wykorzystaniem protokołu UDP

3.4. Warstwa sieciowa (Internetu)

W warstwie sieciowej ramka komunikacyjna obudowywana jest o pola adresowe zawierające adres IP nadawcy oraz odbiorcy wiadomości. Ich obecność w pakiecie jest istotna ze względu na sposób transmisji danych w Internecie, polegający na przekazywaniu pakietu pomiędzy urządzeniami sieciowymi do momentu, aż pakiet trafi do sieci docelowej. Urządzenia sieciowe wiedzą, na który port należy przekazać pakiet adresowany do danej sieci dzięki konfiguracji. Protokół IP służący do adresacji dostępny jest w wersji IPv4 oraz IPv6, różniących się długością nadawanych adresów, sposobem adresacji sieci i podsieci, oraz translacją adresów publicznych na prywatnych w przypadku IPv4 lub jej brakiem w przypadku IPv6.

Oprócz protokołu IP w warstwie sieciowej operują protokoły:

- 1) ARP (ang. *Address Resolution Protocol*) – protokół służący do uzyskania adresu MAC urządzenia, któremu przydzielony jest określony adres IP [11];
- 2) RARP (ang. *Reverse Address Resolution Protocol*) – protokół służący do uzyskania adresu IP urządzenia posiadającego określony adres MAC;
- 3) ICMP (ang. *Internet Control Message Protocol*) – protokół kontrolny pozwalający na nadzorowanie stanu sieci komputerowej.

Konieczność wykorzystania protokołów dostępu do adresu MAC na podstawie adresu IP wynika z różnicy w sposobie adresowania pomiędzy warstwą Internetu, w której wykorzystuje się adresy IP, oraz warstwą dostępu do sieci, w której wykorzystywane są adresy MAC. W trakcie transmisji pakietu każde z pośredniczących urządzeń sieciowych dokonuje dekapsulacji pakietu do postaci dostępnej w warstwie Internetu, a następnie podejmuje decyzję, czy pakiet należy przekazać dalej na podstawie adresu IP. W przypadku konieczności dalszego przekazania pakietu jest on ponownie enkapsulowany informacjami o adresach MAC w warstwie dostępu do sieci. W przeciwnym wypadku pakiet zostaje przekazany do protokołów warstw wyższych. Na rysunku 3.3. przedstawiono schemat warstw, w których przetwarzany jest pakiet w transmisji z dwoma urządzeniami pośredniczącymi.

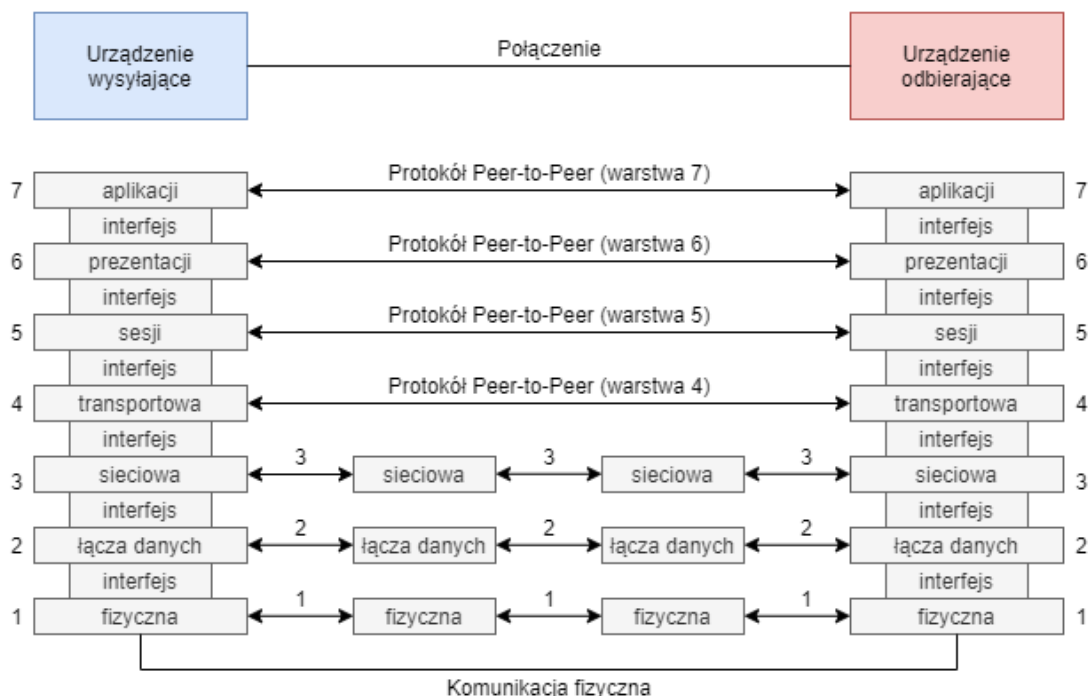


Figure 3.3. Schemat drogi pakietu od nadawcy do odbiorcy w modelu OSI - Computer Notes

3.5. Warstwa interfejsu sieciowego (dostępu do sieci)

Warstwa dostępu do sieci odpowiada za przygotowanie pakietów do transmisji za pomocą medium fizycznego oraz jej przeprowadzenie. Zarówno w przypadku sieci przewodowych jak i bezprzewodowych wyróżnić można w tej części trzy podwarstwy:

- 1) LLC (ang. *Logic Link Control*) – warstwa dołączająca do ramki informacje o wykorzystanym w warstwie transportowej protokole, aby umożliwić urządzeniu docelowemu prawidłową interpretację otrzymanego pakietu;
- 2) MAC (ang. *Media Access Control*) – warstwa tworząca sekcje adresów fizycznych w nagłówku ramki ethernetowej, zapisując adres MAC nadawcy oraz adres MAC odbiorcy lub pośredniczącego routera. W przypadku pośredniczących routerów, adres odbiorcy jest aktualizowany przez router w trakcie przekazywania pakietu;
- 3) Fizyczna (ang. *Physical, PHY*) – składa się z fizycznych komponentów transmisji takich jak np. kable transmisyjne, anteny i nadajniki.

Na poziomie fizycznego interfejsu możliwe są do wyróżnienia dwie główne metody połączenia: przewodowe i bezprzewodowe.

3.5.1. Połączenie przewodowe

Do połączenia przewodowego wykorzystywany jest powszechnie spotykany kabel UTP, wykorzystujący gniazdo 8P8C, często mylnie nazywany RJ-45 ze względu na bardzo zbliżoną konstrukcję (wtyk RJ-45 posiada dodatkową wypustkę) [12]. Stanowi on 8-pinowy wtyk, w którym występują 4 pary oznaczane ciągłym i przerywanym kolorem izolacji. Pozwala on na połączenie dwóch urządzeń sieciowych metodą Point-to-Point, z wykorzystaniem protokołu PPPoE (ang. *Point-to-Point Protocol over Ethernet*). Wtyk 8P8C jest często spotykany zarówno w formie modułów rozszerzających wykorzystujących interfejsy szeregowy jak i zintegrowanego portu na płycie PCB. Rysunek 3.4 przedstawia numerację wyprowadzeń gniazda 8P8C, natomiast rysunek 3.5 numerację linii wtyku 8P8C.



Figure 3.4. Numeracja wyprowadzeń gniazda 8P8C - Electronics Stack Exchange

Połączenie pomiędzy warstwą MAC a warstwą fizyczną odbywa się za pomocą niezależnego od platformy interfejsu MII (ang. *Media-Independent Interface*) lub jego zredukowanej wersji RMII (ang. *Reduced Media-Independent Interface*). Stanowi

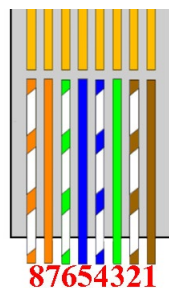


Figure 3.5. Numeracja linii wtyku 8P8C - Electronics Stack Exchange

on połączenie pomiędzy procesorem i sterownikiem portu Ethernet, wyposażonym w przetworniki analogowo-cyfrowe i cyfrowo-analogowe w celu umożliwienia transmisji sygnału różnicowego poprzez kabel Ethernet. Rysunek 3.6 przedstawia schemat funkcjonalny przykładowego sterownika Ethernet, natomiast rysunek 3.7 jego umiejscowienie w schemacie platformy pomiędzy warstwą MAC a warstwą PHY.

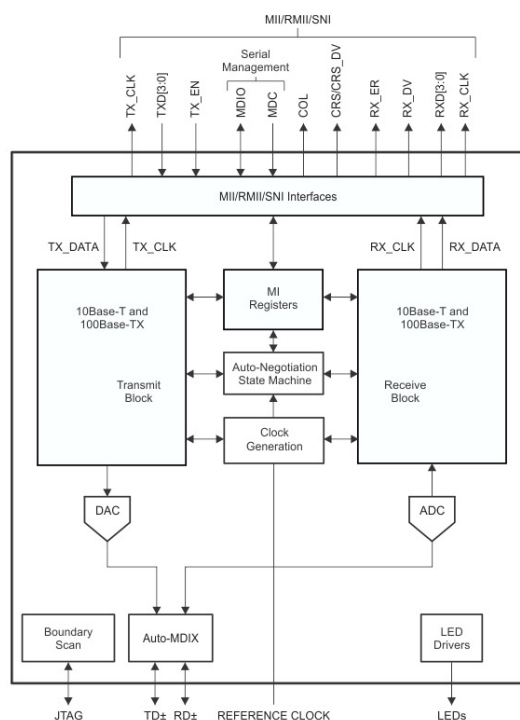


Figure 3.6. Schemat funkcjonalny sterownika Ethernet DB83848-P firmy Texas Instruments - Texas Instruments

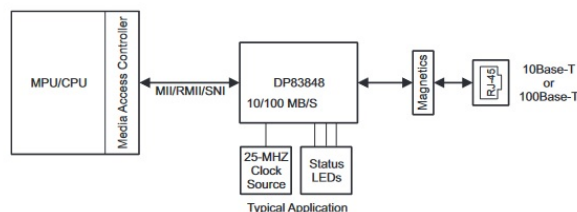


Figure 3.7. Umiejscowienie sterownika DB83848-P - Texas Instruments

Wykorzystanie RMII pozwala na podłączenie różnych warstw fizycznych (np.

kabel UTP, światłowod) do tej samej warstwy łączy danych. Tabela 3.1 przedstawia linie występujące w RMII oraz ich przeznaczenie [13][14]. Skrót MAC oznacza warstwę łączy danych, natomiast skrót PHY warstwę fizyczną.

Table 3.1. Spis linii interfejsu RMII

Oznaczenie linii	Kierunek transmisji	Opis
REF_CLK	dowolny jednokierunkowy	Zegar referencyjny 50 MHz
TXD0	MAC -> PHY	Pierwszy bit nadawczy
TXD1	MAC -> PHY	Drugi bit nadawczy
TX_EN	MAC -> PHY	Sygnał rozpoczęcia transmisji
RXD0	PHY -> MAC	Pierwszy bit odbiorczy
RXD1	PHY -> MAC	Drugi bit odbiorczy
CRS_DV	PHY -> MAC	Linia wykrywania nośnej
RX_ER	PHY -> MAC	Linia błędu odbioru
MDIO	dwukierunkowy	Linia danych administracyjnych
MDC	MAC -> PHY	Sygnał zegarowy linii MDIO

Transmisja za pomocą RMII wymaga konwersji binarnej reprezentacji kodu wysyłanego znaku na notację *big-endian*, której bity o indeksie parzystym (przy numeracji od 0) są przesyłane linią TXD0, natomiast bity o indeksie nieparzystym, linią TXD1, tworząc transmisję za pomocą tzw. di-bitów lub *nibbli*. Podobnie jak w przypadku nadawania, odbiór danych odbywa się poprzez otrzymanie bitów o indeksie parzystym (przy numeracji od 0) za pomocą linii RXD0 oraz bitów o indeksie nieparzystym za pomocą linii RXD1.

Istotną kwestią stanowi połączenie linii wewnątrz kabla UTP do poszczególnych pinów wtyku 8P8C. W przypadku łączenia dwóch urządzeń uznawanych jako *host*, konieczne jest wykorzystanie kabla tzw. krzyżowego (ang. *crossover*), czyli z zamienioną kolejnością pary nadawczej i pary odbiorczej na poszczególnych krańcach kabla. Natomiast w przypadku podłączenia urządzenia typu *host* do urządzenia takiego jak router (posiadający wbudowany przełącznik), należy wykorzystać kabel prosty (ang. *straight-thru*). Sposoby zaciskania wtyku 8P8C na liniach kabla UTP znane są jako T-568A oraz T-568B. Za kabel krzyżowy uznaje się kabel UTP posiadający różne ułożenie przewodów na końcach, natomiast za kabel prosty uznawany jest kabel o tym samym ułożeniu przewodów na obu końcach.

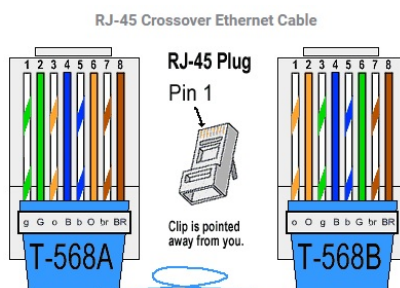


Figure 3.8. Ułożenia przewodów w kablu UTP - The Internet Centre Inc.

3.5.2. Połączenie bezprzewodowe

Ze względu na popularność integracji systemów wbudowanych z sieciami bezprzewodowymi, w szczególności w przypadku automatyki domowej i biurowej, na rynku dostępne jest wiele gotowych modułów oferujących funkcjonalności sieci bezprzewodowych, zarówno typu Wi-Fi jak i danych komórkowych. Ze względu na ich bardzo zbliżoną budowę, w której różnice funkcjonalne sprowadzają się często jedynie do rodzaju anteny w związku z obsługą różnych częstotliwości, zdecydowano się na zbiorczy opis w/w modułów.

Większość modułów występuje w formie preprogramowanych układów cyfrowych na płycie drukowanej PCB, najczęściej z wbudowaną anteną mikropaskową (w postaci ścieżki na płycie drukowanej) lub wyprowadzonym wyjściem antenowym SMA, wykorzystywanym dla przewodów koncentrycznych, wyjściem IPEX, lub wyjściem u.FL. Rysunek 3.7 przedstawia przykłady anten wykorzystywanych przez moduły bezprzewodowe.

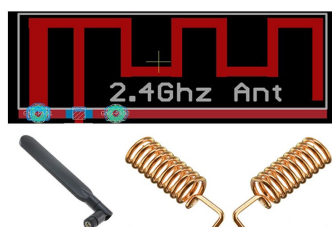


Figure 3.9. Rodzaje anten dla urządzeń systemów wbudowanych - Circuit Digest

Część układów implementujących interfejsy bezprzewodowe utworzonych jest jako rozszerzenia typu Shield, umożliwiając bardzo łatwą integrację z platformą sprzętową Arduino bez konieczności lutowania. Budowa takiego modułu jest bardzo zbliżona do konstrukcji platformy sprzętowej, posiadając wyprowadzenia zgodne co do funkcji i położenia z wyprowadzeniami obecnymi na wybranej płycie. W przypadku modułów oferujących dostęp do sieci GSM konieczne jest dodatkowe uzupełnienie go o gniazdo na kartę SIM, podobnie jak w przypadku telefonów komórkowych.

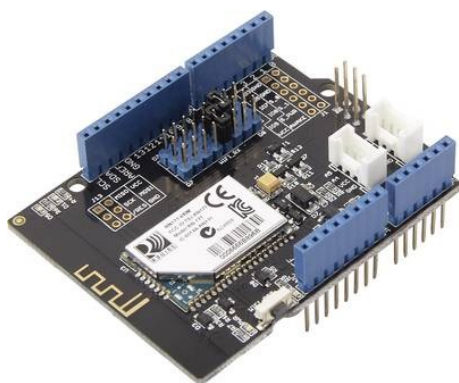


Figure 3.10. Shield Wi-Fi - Seeed Studio



Figure 3.11. Shield GSM - Botland

3.6. Implementacje stosu TCP/IP

Implementacja stosu TCP/IP, zarówno ze względu na ilość jak i złożoność protokołów, stanowi złożone zadanie, w związku z czym najpopularniejszym wariantem jest skorzystanie z gotowych rozwiązań. Część urządzeń (w szczególności moduły rozszerzające) implementują cały stos TCP/IP, wymagając od użytkownika jedynie implementacji komunikacji z wykorzystaniem wybranego lokalnego interfejsu szeregowego, jak np. SPI lub I²C.



Figure 3.12. Moduł ENC28J60 - Aliexpress

Jako przykład rozwiązania zapewniającego zewnętrzny interfejs Ethernet można podać moduł z układem ENC28J60 [15], przedstawiony na rys. 3.12. Jest on kompatybilny z sieciami 10/100/-1000Base-T, wspierając w szczególności transmisję 10Base-T. Do komunikacji z mikrokontrolerem wykorzystuje on interfejs SPI o częstotliwości zegara 20 MHz. Oferuje on implementację jedynie warstwy fizycznej oraz łącza danych modelu OSI, co wymusza na użytkowniku implementację pozostałych warstw stosu TCP/IP. Posiada on 8 kB programowalnej pamięci na bufor odbioru i nadawania oraz sprzętowe wyznaczanie sumy kontrolnej. Ze względu na implementację warstwy aplikacji po stronie mikrokontrolera, ilość możliwych do wykorzystania gniazd (ang. *socket*) zależy od ilości pamięci RAM mikrokontrolera.

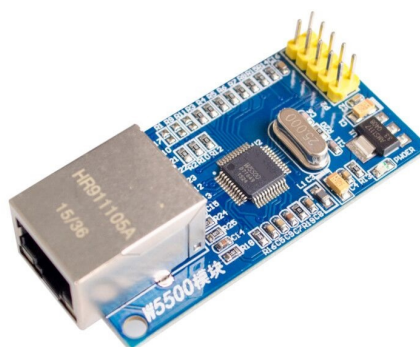


Figure 3.13. Moduł W5500 - Aliexpress

Drugim przykładem jest układ W5500 przedstawiony na rys. 3.13. W przeciwieństwie do układu ENC28J60, oferuje on także część protokołów warstw wyższych, takich jak TCP, UDP, ICMP, IPv4, ARP, IGMP i PPPoE [16]. Ponadto oferuje on

także funkcjonalność *wake on LAN*. Układ ten dostępny jest m.in. w postaci platformy Shield dla płytek Arduino. Działa on ze standardami 10/100 Base-T. Posiada on 32 kB programowalnej pamięci dla buforów odbierania/nadawania i wspiera jednocześnie 8 gniazd. Pracuje on z poziomem logicznym 3,3 V, posiadając tolerancję dla 5 V na pinach wejścia/wyjścia (ang. *Input/Output*, I/O). Tabela 3.2 przedstawia porównanie w/w modułów wraz z orientacyjnymi cenami.

Table 3.2. Zestawienie parametrów modułów Ethernet

Moduł	Standard Transmisji	Protokoły	Interfejs	Prędkość Interfejsu	Cena
ENC28J60	10Base-T	MAC	SPI	20 MHz	32,70 zł
W5500	10/100Base-T	MAC, TCP, UDP, ICMP, IPv4, ARP,	SPI IGMP, PPPoE	80 MHz	115,00 zł

Dwoma popularnymi modułami bezprzewodowymi są moduły ESP8266 oraz ESP32, przedstawione odpowiednio na rys 3.14 oraz rys 3.15. Drugi z wymienionych modułów stanowi nowszą wersję pierwszego. Oba moduły pracują w częstotliwości 2,4 GHz, w standardzie IEEE 802.11 b/g/n. Różnią się one dostępnymi prędkościami, zabezpieczeniami oraz interfejsami szeregowymi umożliwiającymi podłączenie do mikrokontrolera. Oba moduły wykorzystują jako złącza pady i złącza THT, umożliwiające wlutowanie pinów typu goldpin. Zarówno ESP8266 jak i ESP32 potrafią pracować w trybie miękkiego punktu dostępowego (ang. *Soft Access Point*, softAP) jak i stacji nadawczej (ang. *Station*, STA).

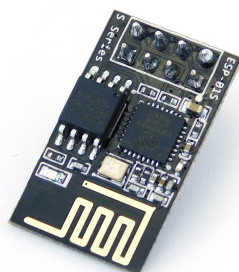


Figure 3.14. Moduł ESP8266 - Nettigo



Figure 3.15. Moduł ESP32 - Botland

ESP8266 umożliwia komunikację z prędkością do 72,2 Mbps. *Firmware* udostępniany przez producenta układu realizuje implementację protokołów IPv4, TCP, UDP, HTTP, ICMP i MAC oraz oferuje szyfrowanie WEP, WPA/WPA2, TKIP, a także AES [17]. Do komunikacji z mikrokontrolerem wykorzystywany jest interfejs UART. Do sterowania modułem wykorzystywane są komendy AT. Moduł pracuje z napięciem w okolicach 3,3 V oraz dostępny jest w dwóch wariantach anten – z anteną mikropasową lub złączem u.FL.

ESP32 stanowi nowszą wersję w/w ESP8266. W układzie zaimplementowano dwurdzeniowy procesor, co pozwoliło na zwiększenie prędkości transmisji do 150 Mbps [18]. Podobnie jak w przypadku ESP8266, dla układu dostępny jest *firmware* implementujący protokoły IPv4, TCP, UDP, HTTP, ICMP i MAC, lecz posiadający

bardziej rozbudowane zabezpieczenia, rozszerzając dostępne protokoły szyfrowania o PSK/Enterprise, SHA-2, ECC i RSA-4096. Niektóre warianty posiadają wbudowaną możliwość obsługi protokołu IPv6. Istnieje możliwość przygotowania przez użytkownika własnej wersji *firmware'u* zawierającego stosownie skonfigurowane protokoły przy pomocy SDK (ang. *Software Development Kit*) udostępnionego przez producenta. Moduł pracuje z napięciem w okolicach 3,3 V. Do komunikacji z urządzeniem wykorzystywane są interfejsy SPI, I²C oraz UART. Tabela 3.3 przedstawia zestawienie parametrów wymienionych modułów wraz z orientacyjnymi cenami.

Table 3.3. Zestawienie parametrów modułów bezprzewodowych

Moduł	Standard Transmisji	Protokoły	Interfejs	Prędkość Transmisji	Cena
ESP8266	802.11 b/g/n	MAC, TCP, UDP, HTTP, IPv4, ICMP	UART	72,2 Mbps	14,90 zł
ESP32	802.11 b/g/n	MAC, TCP, UDP, HTTP, IPv4, (IPv6), ICMP	SPI, I ² C, UART	150 Mbps	127,50 zł

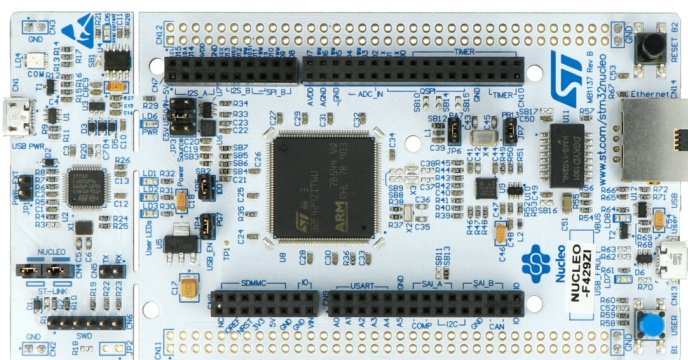


Figure 3.16. Platforma sprzętowa Nucleo-F429ZI posiadająca złącze Ethernet - Botland

Niektórzy producenci platform sprzętowych decydują się na jedynie częściową implementację stosu TCP/IP dla swoich platform w przypadku, gdy jest ona wyposażona w interfejs sieciowy (np. gniazdo 8P8C). Przykładem tego są platformy sprzętowe Nucleo-F4 (m.in. model Nucleo-F429ZI przedstawiony na rys. 3.16.) wyposażone w w/w gniazdo. Oprogramowanie HAL (ang. *Hardware Abstract Layer*) udostępniane przez firmę ST Microelectronics dla wspomnianych mikrokontrolerów zapewnia implementację warstwy dostępu do sieci stosu TCP/IP, jednak nie oferuje wyższych warstw, wymagając od użytkownika wykorzystania oprogramowania pośredniego (ang. *middleware*) takiego jak m.in. otwarte stosy LwIP (ang. *Lightweight IP*) oraz uIP (ang. *micro IP*). W celu wykorzystania wymienionych stosów protokołów należy wykonać wcześniej ich przeniesienie (ang. *port*) na dany mikrokontroler lub skorzystać z gotowego rozwiązania, jeśli zostało ono przygotowane.

Stos LwIP stanowi implementację protokołów warstw aplikacji, transportowej i Internetu. Oferuje on następujące protokoły [19]:

- 1) IP;
- 2) ICMP;
- 3) UDP;
- 4) DHCP;
- 5) TCP;
- 6) ARP;
- 7) BSD (opcjonalnie)

Implementacja stosu LwIP od wersji v.2.0.0 wspiera adresację zarówno za pomocą IPv4 jak i IPv6.

uIP stanowi otwartą implementację stosu TCP/IP dedykowaną dla mikrokontrolerów o bardzo dużych ograniczeniach związanych z pamięcią Flash oraz RAM. Zgodnie z treścią dokumentu [20] stos uIP potrafi minimalizować użycie RAMu do kilkuset bajtów. Implementuje on następujące protokoły:

- 1) TCP;
- 2) UDP;
- 3) IP;
- 4) ICMP;
- 5) ARP

W związku z bardzo dużymi wymaganiami optymalizacji pamięci, wszystkie protokoły warstwy aplikacji są pozostawione do implementacji przez użytkownika. Wynika to również z braku możliwości przewidzenia w jakich aplikacjach stos zostanie wykorzystany.

Rozdział 4

Praktyczna implementacja interfejsu sieciowego

4.1. Założenia projektowe

Podczas opracowywania projektu praktycznego rozwiązania podjęto następujące założenia projektowe:

- 1) Zaimplementowany zostanie interfejs sieciowy przewodowy wykorzystujący złącze Ethernet oraz interfejs sieciowy bezprzewodowy wykorzystujący moduł ESP8266.
- 2) Moduł IoT będzie otrzymywał informacje sieciowe, takie jak swój adres IP, maskę sieci oraz adres IP bramy domyślnej za pomocą usługi DHCP.
- 3) Przygotowane rozwiązanie będzie pozwalać na nawiązanie komunikacji z wykorzystaniem polecenia *ping*.
- 4) Przygotowane rozwiązanie oferować będzie serwer echo odsyłający otrzymane przez moduł IoT wiadomości za pomocą protokołu TCP z powrotem do nadawcy.

4.2. Wybrane komponenty

4.2.1. Platforma sprzętowa

Ze względu na gotową realizację części fizycznej oraz dużą dostępność materiałów pomocniczych w postaci dokumentacji i zasobów społeczności, jako platformę sprzętową wybrano płytke Nucleo-F429ZI firmy ST Microelectronics. Do implementacji interfejsu bezprzewodowego wykorzystano omówiony we wcześniejszej części pracy moduł ESP8266, ze względu na jego niewielki koszt.

4.2.2. Środowisko i język programowania

Podczas analizy dostępnych zasobów zdecydowano się na wybór dedykowanego dla wybranej platformy sprzętowej środowiska STM32CubeIDE. Wynika to z dostępno-

ści wbudowanych narzędzi konfiguracyjnych umożliwiających ustawienie wybranych parametrów platformy sprzętowej bez konieczności przechodzenia na poziom manipulacji rejestrami mikrokontrolera. Jako język programowania wybrano język C.

4.2.3. Metody weryfikacji działania systemu

Weryfikacja działania systemu odbędzie się w sposób dwuetapowy. Pierwszym etapem weryfikacji poprawności funkcjonowania rozwiązania jest przesłanie informacji na temat modułu IoT, takich jak jego adres IP, maska sieci oraz adres bramy domyślnej poprzez wirtualny port szeregowy. Drugi etap stanowi próba komunikacji z modułem IoT z wykorzystaniem polecenia *ping*, oraz poprzez komunikację z zaimplementowanym serwerem echo z użyciem programu Hercules SETUP Utility umożliwiającego komunikację z wykorzystaniem protokołu TCP.

4.3. Implementacja części sprzętowej

4.3.1. Interfejs Ethernet

Ze względu na wybraną platformę sprzętową, implementacja części sprzętowej została wykonana fabrycznie na płytce Nucleo-F429ZI. Płytkę posiada wlutowane złącze 8P8C z podłączonym interfejsem RMII, przedstawione na rys. 4.1.

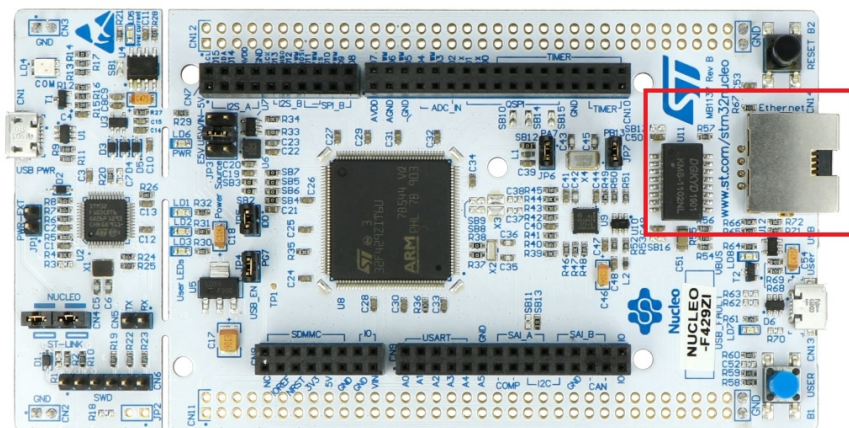


Figure 4.1. Złącze 8P8C obecne na płytce Nucleo-F429ZI

4.3.2. Interfejs bezprzewodowy

Do komunikacji z układem ESP8266 wykorzystano interfejs USART6 mikrokontrolera STM32-F429ZI, obecnego na wyprowadzeniach PC6 oraz PC7. Dodatkowo wykorzystano wyprowadzenie PB3 do podania stanu wysokiego na wejściu *enable* modułu ESP8266. Rysunek 4.2 przedstawia schemat połączeń.

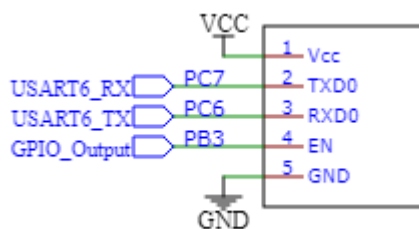


Figure 4.2. Schemat podłączenia modułu ESP8266

4.4. Implementacja części programowej

4.4.1. Interfejs Ethernet

Pierwszym etapem implementacji rozwiązania była inicjalizacja interfejsu Ethernet umożliwiając prawidłowe przesłanie polecenia *ping* oraz sygnalizację stanu połączenia za pomocą diody LED.

Proces inicjalizacji rozpoczęty został od uruchomienia środowiska STM32CubeIDE i utworzenia w nim nowego projektu wybierając mikrokontroler STM32L429ZI. Rys. 4.3 przedstawia wybór platformy mikrokontrolera, natomiast rys. 4.4. opcje tworzonego projektu.

Part No.	Reference	Marketing	Unit Price f...	Board	Package	Flash	RAM	ID	Freq.
STM32F429ZI	STM32F429...	Active	6.48	NUCLEO-STIM32	LQFP144	2048 kBytes	256 kBytes	114	180 MHz
STM32F429ZI	STM32F429...	Active	6.48		WLCSP143	2048 kBytes	256 kBytes	114	180 MHz

Figure 4.3. Wybór mikrokontrolera w STM32CubeIDE

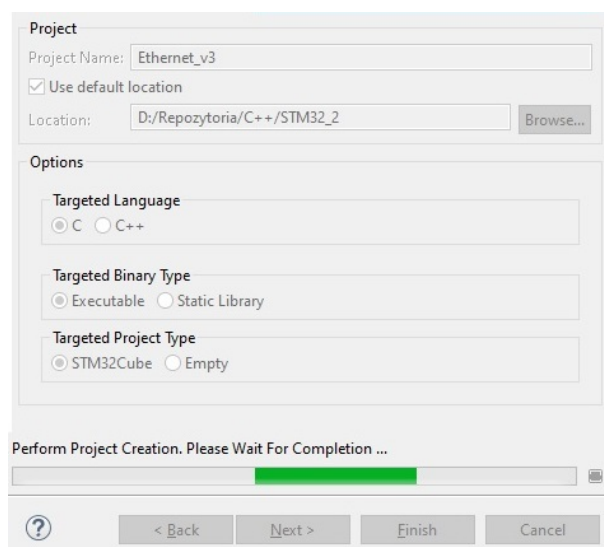


Figure 4.4. Utworzenie projektu

Jako zegar został wykorzystany sygnał zegarowy dostarczony przez moduł ST-LINK stanowiący część płytki Nucleo-F429ZI. W tym celu należało przestawić High Speed Clock w zakładce RCC w tryb BYPASS, co przedstawiono na rys. 4.5. Wynikowa konfiguracja sygnału zegarowego została przedstawiona na rys. 4.6.

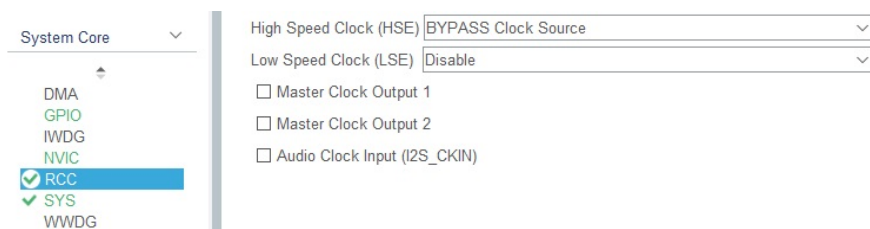


Figure 4.5. Przełączenie zegaru HSE w tryb BYPASS

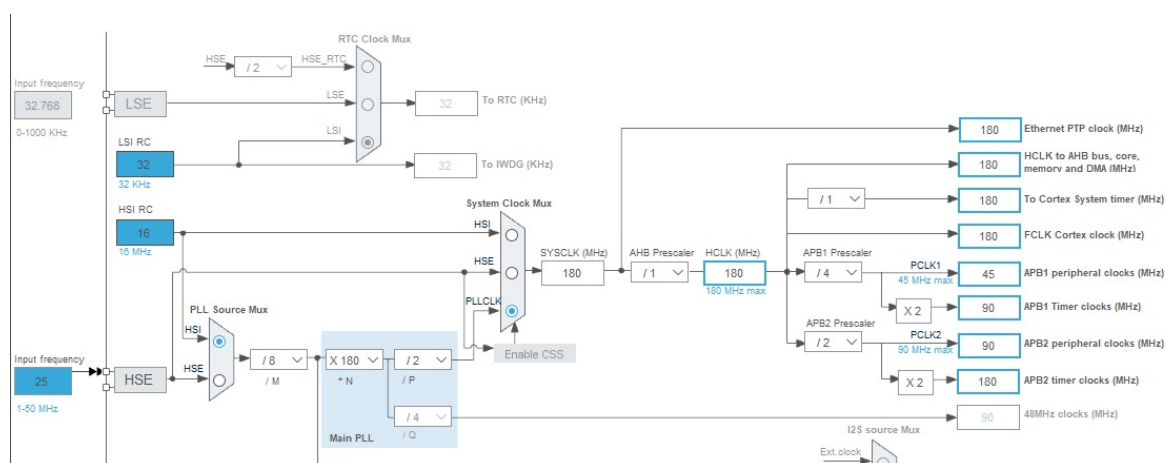


Figure 4.6. Konfiguracja sygnału zegarowego po przestawieniu HSE w tryb BYPASS

W zakładce *Connectivity* należy wstępnie skonfigurować interfejs Ethernet ustawiając jego tryb na RMII. Wynika to z konstrukcji płytki Nucleo-429ZI w której do połączenia interfejsu Ethernet wykonane jest przy pomocy interfejsu RMII. Domyślny adres MAC urządzenia wynosi 00:80:E1:00:00:00. W trakcie ustawiania domyślnych wartości przez konfigurator, adres PHY urządzenia przypisany został na 1. Należy go zmienić na 0. Wyniki powyższych działań przedstawiono na rys. 4.7. i 4.8.

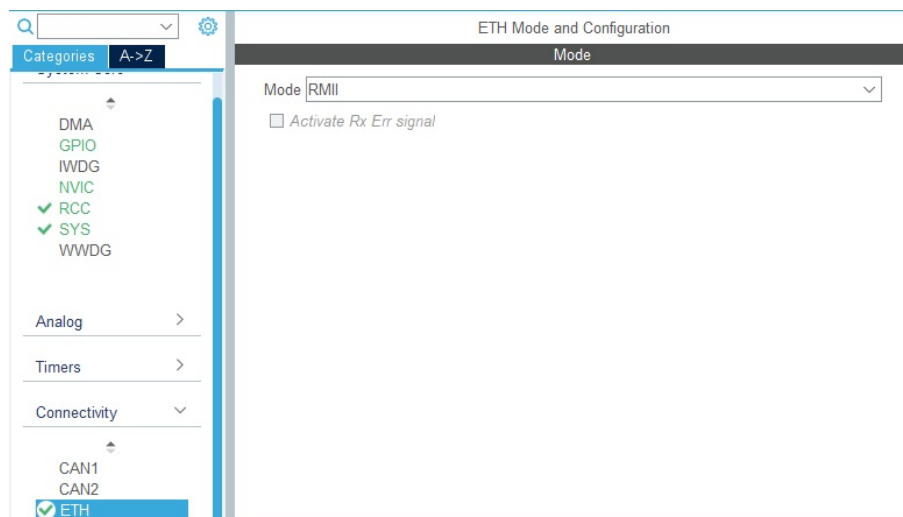


Figure 4.7. Przypisanie trybu RMII do interfejsu Ethernet

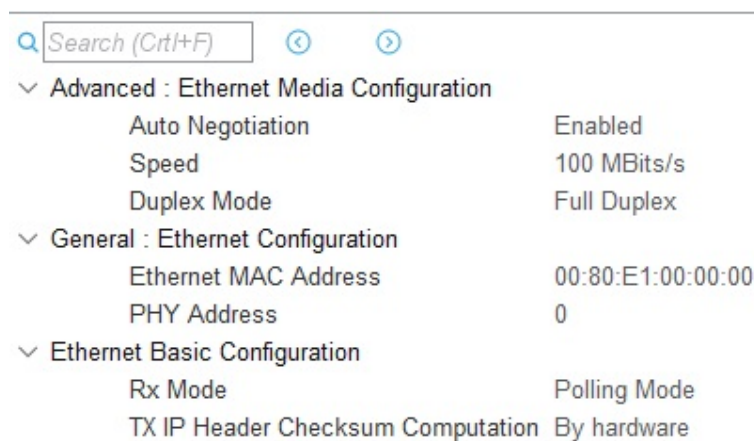


Figure 4.8. Docelowe parametry interfejsu RMII

W związku z połączeniami występującymi na płytce Nucleo-F429ZI zgodnie z tabelą przedstawioną w dokumentacji producenta [21] piny wyjściowe mikrokontrolera zostały przypisane w sposób przedstawiony w tabeli 4.1. Schemat połączeń interfejsu RMII przedstawiony jest na rys. 4.9.

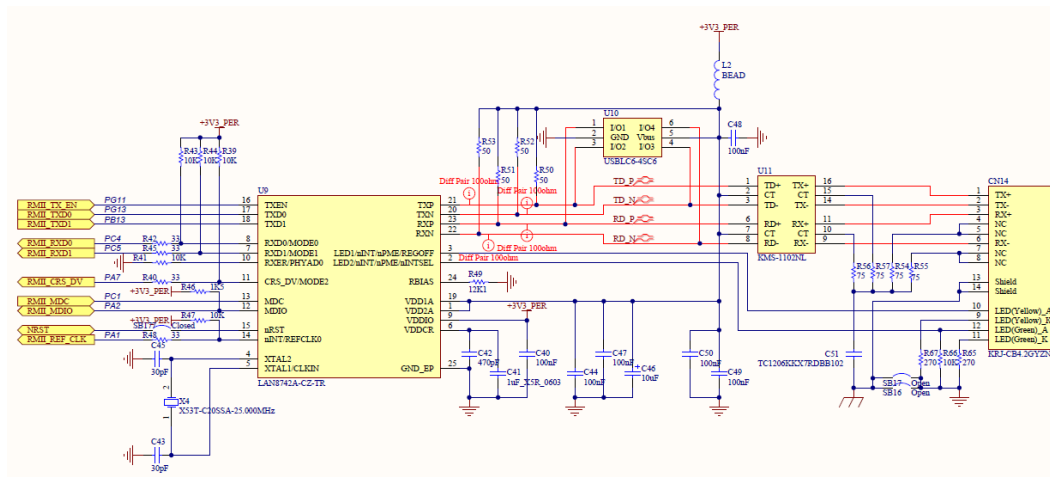


Figure 4.9. Schemat podłączenia interfejsu RMII

Table 4.1. Przyporządkowanie linii wyjściowych mikrokontrolera

Nazwa pinu	Funkcja
PA1	ETH_REF_CLK
PA2	ETH_MDIO
PA7	ETH_CRS_DV
PC4	ETH_RXD0
PC5	ETH_RXD1
PG11	ETH_TX_EN
PG13	ETH_TXD0
PB13	ETH_TXD1
PB14	Dioda LED Czerwona
PB0	Dioda LED Zielona
PB7	Dioda LED Niebieska
Nazwa pinu	Funkcja
PD9	Linia RX wirtualnego portu szeregowego USART3
PD8	Linia TX wirtualnego portu szeregowego USART3

Kolejny krok stanowiła konfiguracja stosu LwIP wykorzystywanego do obsługi protokołów stosu TCP/IP. Aby tego dokonać należy przejść do zakładki *Middleware* jak przedstawiono na rys. 4.10, w której użytkownik otrzymuje dostęp do różnego rodzaju oprogramowania dla wybranego mikrokontrolera, jak np. stos LwIP lub system czasu rzeczywistego freeRTOS. Stos LwIP charakteryzuje się bardzo dużą ilością dostępnych opcji konfiguracyjnych. Na potrzeby implementacji realizowanego w ramach pracy rozwiązania, większość konfiguracji można pozostawić jako domyślną. Należy się jednak upewnić, iż uruchomiona jest opcja LWIP_NETIF_LINK_CALLBACK, stanowiąca funkcję wołaną w momencie wykrycia podłączenia wtyczki Ethernet do gniazda 8P8C, jak pokazano na rys. 4.11.

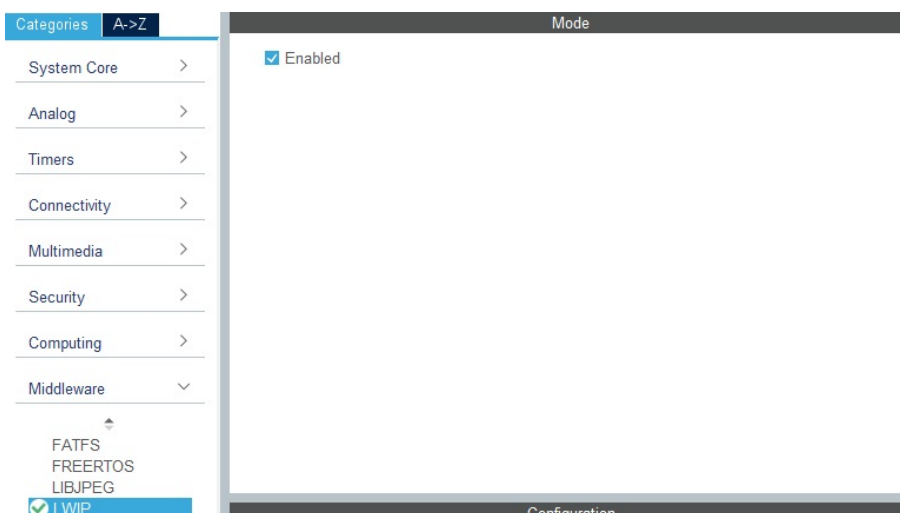


Figure 4.10. Uruchomienie stosu LwIP

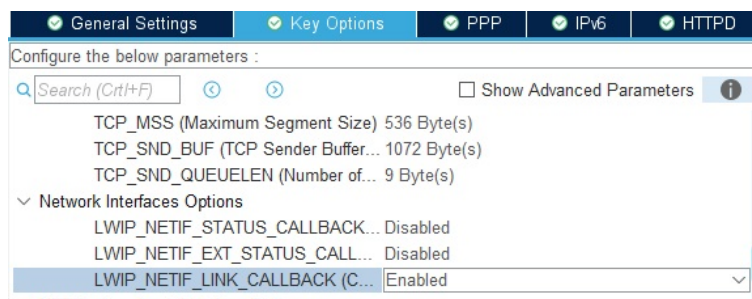


Figure 4.11. Opcja LWIP_NETIF_LINK_CALLBACK

Korzystając z zakładki *Clock Configuration*, zegar taktujący interfejs Ethernet został ustawiony na częstotliwość 180 MHz, za pomocą pętli PLL i odpowiednich modyfikatorów (ang. *prescaler*), jak przedstawiono na rysunku 4.12.

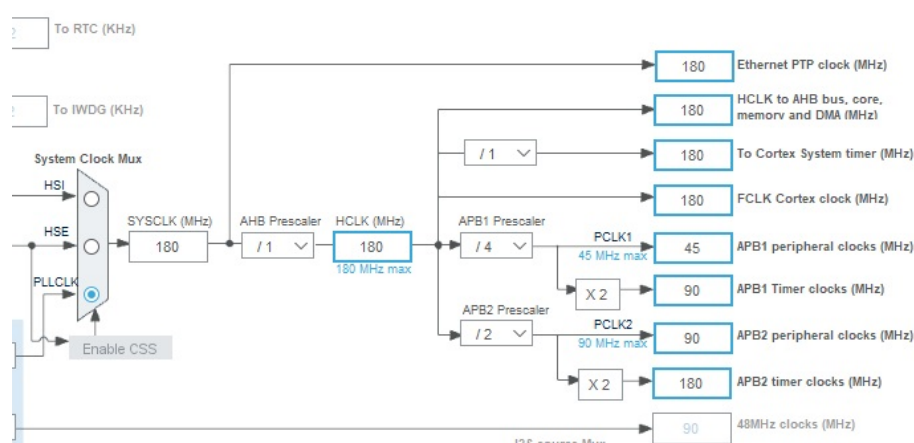


Figure 4.12. Konfiguracja zegarów

Kolejny krok stanowiło wygenerowanie kodu oraz jego stosowna modyfikacja. Podczas implementacji serwera echo z wykorzystaniem stosu LwIP oraz modyfikacji funkcji *MX_LWIP_Process()* wykorzystano kod z przykładowej implementacji autorstwa koreańskiego użytkownika blogu Naver o pseudonimie eziya76 [22].

Listing 4.1. Funkcja *main()* implementacji interfejsu Ethernet

```
1  /* Struktura połączenia */
2  extern struct netif gnetif;
3  extern UART_HandleTypeDef huart3;
4
5  /* Bufory tekstowe na dane połączenia */
6  char conn_msg_str[24]; // wiadomość
7  char ip_str[24];      // adres IP
8  char mask_str[24];    // maska
9  char gw_str[24];      // brama
10 char mac_str[30];      // adres MAC
11
12 int main(void)
13 {
14     /* USER CODE BEGIN 1 */
15
16     /* USER CODE END 1 */
17
18     /* MCU Configuration-----*/
19
20     /* Reset of all peripherals, Initializes the Flash interface and the
21      * SysTick. */
22     HAL_Init();
23
24     /* USER CODE BEGIN Init */
25
26     /* USER CODE END Init */
27
28     /* Configure the system clock */
29     SystemClock_Config();
30
31     /* USER CODE BEGIN SysInit */
32
33     /* USER CODE END SysInit */
34
35     /* Initialize all configured peripherals */
36     MX_GPIO_Init();
37     MX_USART3_UART_Init();
38     MX_LWIP_Init();
39     /* USER CODE BEGIN 2 */
40     /* Przesłanie adresu MAC za pomocą portu szeregowego */
41     send_MAC_address();
42     /* Sprawdzenie stanu połączenia */
43     notify_dhcp_status(&gnetif);
44     /* Inicjalizacja serwera echo */
45     init_echo();
46     /* USER CODE END 2 */
47
48     /* Infinite loop */
49     /* USER CODE BEGIN WHILE */
50     while (1)
51     {
52         /* Przetworzenie danych przez stos LwIP */
53         MX_LWIP_Process();
54         /* Sprawdzenie stanu połączenia */
55         notify_dhcp_status(&gnetif);
56         /* USER CODE END WHILE */
57
```

```
58      /* USER CODE BEGIN 3 */
59  }
60      /* USER CODE END 3 */
61 }
```

Listing 4.1 przedstawia implementację funkcji *main()* dla projektu implementującego interfejs Ethernet. Część funkcji została wygenerowana poprzez narzędzie do generowania kodu na podstawie konfiguracji środowiska STM32CubeIDE (części oznaczone komentarzami w języku angielskim).

Utworzone zostało 5 globalnych buforów tekstowych służących do przechowywania i formatowania informacji o połączeniu takich jak adres IP, maska sieci i adres bramy, adresu MAC oraz informacji o statusie połączenia. Informacje o połączeniu, informacja o statusie połączenia oraz adres MAC przesyłane są do komputera nadzorującego za pomocą portu szeregowego USART3 wewnątrz funkcji *notify_dhcp_status()*.

Funkcja *MX_LWIP_Process()* odpowiada za otrzymanie dynamicznego adresu IP z serwera DHCP uruchomionego na routerze i została przedstawiona na listingu 4.2. Funkcja *init_echo()* inicjalizuje serwer echo, odsyłający kopię otrzymanej wiadomości do nadawcy. Listing 4.3 przedstawia implementację funkcji *notify_dhcp_status()*. Listingi 4.4 - 4.7 przedstawiają implementacje funkcji przekształcających informacje o połączeniu i adresie MAC na podstać ciągu znakowego.

Listing 4.2. Funkcja *MX_LWIP_Process*

```
1 void MX_LWIP_Process(void)
2 {
3     /* USER CODE BEGIN 4_1 */
4     /* USER CODE END 4_1 */
5     ethernetif_input(&gnetif);
6
7     /* USER CODE BEGIN 4_2 */
8     /* USER CODE END 4_2 */
9     /* Handle timeouts */
10    sys_check_timeouts();
11
12    /* USER CODE BEGIN 4_3 */
13    ethernetif_set_link(&gnetif);
14    /* Jeżeli kabel sieciowy jest podłączony, i nie ustawiono połączenia */
15    if(netif_is_link_up(&gnetif) && ! netif_is_up(&gnetif))
16    {
17        /* Ustawienie połączenia */
18        netif_set_up(&gnetif);
19        /* Pobór adresu z DHCP routera */
20        dhcp_start(&gnetif);
21    }
22    /* USER CODE END 4_3 */
23 }
```

Listing 4.3. Funkcja nadzoru stanu połączenia

```

1  /* Funkcja nadzorująca otrzymanie adresu z DHCP */
2  void notify_dhcp_status( struct netif *netif)
3  {
4      /* Flaga statusu połączenia */
5      static uint8_t connected = 0;
6
7      /* Jeżeli kabel sieciowy jest podłączony, i adres IP jest różny od 0 */
8      if (netif_is_link_up(netif) && netif->ip_addr.addr != 0)
9      {
10         /* Jeżeli wcześniej stan połączenia był rozłączony */
11         if (!connected)
12         {
13             /* Przygotowanie wiadomości o połączeniu */
14             sprintf(conn_msg_str, "CONNECTED\n\r");
15             /* Przygotowanie wiadomości z adresem IP */
16             parse_ip(netif->ip_addr.addr);
17             /* Przygotowanie wiadomości z maską sieci */
18             parse_mask(netif->netmask.addr);
19             /* Przygotowanie wiadomości z adresem bramy */
20             parse_gateway(netif->gw.addr);
21             /* Ustawienie flagi połączenia */
22             connected = 1;
23             /* Przesłanie poszczególnych wiadomości za pomocą portu
24              * szeregowego */
25             HAL_UART_Transmit(&huart3, (unsigned char*)conn_msg_str,
26                               sizeof(conn_msg_str), 10);
27             HAL_UART_Transmit(&huart3, (unsigned char*)ip_str,
28                               sizeof(ip_str), 10);
29             HAL_UART_Transmit(&huart3, (unsigned char*)mask_str,
30                               sizeof(mask_str), 10);
31             HAL_UART_Transmit(&huart3, (unsigned char*)gw_str,
32                               sizeof(gw_str), 10);
33         }
34         /* Zgaszenie czerwonej diody i zapalenie zielonej diody jako
35          * sygnalizacja nawiązania połączenia */
36         HAL_GPIO_WritePin(LD1_GPIO_Port, LD1_Pin, GPIO_PIN_SET);
37         HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
38     }
39     /* W przypadku odłączenia kabla sieciowego lub nie otrzymania adresu
40      * z DHCP routera */
41     else
42     {
43         /* Jeżeli wcześniej stan połączenia był podłączony */
44         if (connected)
45         {
46             /* Przygotowanie wiadomości o rozłączeniu */
47             sprintf(conn_msg_str, "DISCONNECTED\n\r");
48             /* Zresetowanie adresu IP */
49             netif->ip_addr.addr = 0;
50             /* Zresetowanie flagi połączenia */
51             connected = 0;
52             /* Przesłanie wiadomości o rozłączeniu za pomocą portu
53              * szeregowego */
54             HAL_UART_Transmit(&huart3, (unsigned char*)conn_msg_str,
55                               sizeof(conn_msg_str), 10);
56         }
57         /* Zgaszenie zielonej diody i zapalenie czerwonej diody jako

```

```
58     * sygnalizacja rozłączenia */
59     HAL_GPIO_WritePin(LD1_GPIO_Port, LD1_Pin, GPIO_PIN_RESET);
60     HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
61 }
62 }
```

Listing 4.4. Funkcja przekształcająca adres IP na postać ciągu znakowego

```
1  /* Funkcja zamieniająca adres IP na ciąg znakowy */
2  void parse_ip(uint32_t ip)
3  {
4      /* Bufor na części adresu IP */
5      uint16_t ip_part[4];
6      /* Wyciągnięcie poszczególnych części adresu IP
7       * poprzez operacje bitowe */
8      for(uint8_t i = 0; i < 4; i++)
9      {
10         ip_part[3 - i] = (ip >> (i * 8)) & 0xFF;
11     }
12     /* Sformatowanie adresu IP do ciągu znakowego */
13     sprintf(ip_str, "IP: %hu.%hu.%hu.%hu\n\r", ip_part[3],
14             ip_part[2], ip_part[1], ip_part[0]);
15 }
```

Listing 4.5. Funkcja przekształcająca maskę sieci na postać ciągu znakowego

```
1  /* Funkcja zamieniająca maskę na ciąg znakowy */
2  void parse_mask(uint32_t msk)
3  {
4      /* Bufor na części maski */
5      uint16_t msk_part[4];
6      /* Wyciągnięcie części maski za pomocą operacji bitowych */
7      for(uint8_t i = 0; i < 4; i++)
8      {
9         msk_part[3 - i] = (msk >> (i * 8)) & 0xFF;
10     }
11     /* Sformatowanie maski do postaci ciągu znakowego */
12     sprintf(mask_str, "Msk: %hu.%hu.%hu.%hu\n\r", msk_part[3],
13             msk_part[2], msk_part[1], msk_part[0]);
14 }
```

Listing 4.6. Funkcja przekształcająca adres bramy na postać ciągu znakowego

```
1  /* Funkcja zamieniająca bramę na ciąg znakowy */
2  void parse_gateway(uint32_t gw)
3  {
4      /* Bufor na części bramy */
5      uint16_t gw_part[4];
6      /* Wyciągnięcie części bramy za pomocą operacji bitowych */
7      for(uint8_t i = 0; i < 4; i++)
8      {
9         gw_part[3 - i] = (gw >> (i * 8)) & 0xFF;
10     }
11     /* Sformatowanie bramy do postaci ciągu znakowego */
12     sprintf(gw_str, "Gw: %hu.%hu.%hu.%hu\n\r", gw_part[3],
13             gw_part[2], gw_part[1], gw_part[0]);
14 }
```

Listing 4.7. Funkcja przesyłająca adres MAC na początku pracy urządzenia

```

1  /* Funkcja przesyłająca adres MAC urządzenia */
2  void send_MAC_address(void)
3  {
4      /* Offset dla znaków heksadecymalnych */
5      uint8_t hex_offset = 'A' - 10;
6      /* Offset dla znaków dziesiętnych */
7      uint8_t dec_offset = '0';
8      /* Bufor znaków adresu MAC */
9      char mac_part[12];
10
11     /* Iteracja przez adres MAC */
12     for(uint8_t i = 0; i < 6; i++)
13     {
14         /* Wyciągnięcie wartości znaków adresu MAC */
15         mac_part[i*2] = (gnetif.hwaddr[i] & 0xF0) >> 4;
16         mac_part[i*2+1] = gnetif.hwaddr[i] & 0x0F;
17
18         /* Sformatowanie otrzymanych wartości do postaci znakowych */
19         if(mac_part[i*2] > 10) mac_part[i*2] += hex_offset;
20         else mac_part[i*2] += dec_offset;
21
22         if(mac_part[i*2+1] > 10) mac_part[i*2+1] += hex_offset;
23         else mac_part[i*2+1] += dec_offset;
24     }
25     /* Sformatowanie znaków do wiadomości z adresem MAC */
26     sprintf(mac_str, "MAC:␣%c%c:%c%c:%c%c:%c%c:%c%c:\n\n\r",
27             mac_part[0], mac_part[1],
28             mac_part[2], mac_part[3],
29             mac_part[4], mac_part[5],
30             mac_part[6], mac_part[7],
31             mac_part[8], mac_part[9],
32             mac_part[10], mac_part[11]);
33     /* Przesłanie adresu MAC za pomocą portu szeregowego */
34     HAL_UART_Transmit(&huart3, (unsigned char*)mac_str, sizeof(mac_str), 10);
35 }

```

Do obsługi serwera echo utworzona została struktura *echo_info* przechowująca informacje dotyczące stanu połączenia, ilości retransmisji, wskaźnik na strukturę pcb stosu LwIP, oraz wskaźnik na bufor odbiorczo/nadawczy. Oprócz tego utworzony został typ wyliczeniowy pozwalający na posługiwanie się nazwami stanów połączenia zamiast wartościami numerycznymi. Wymienione wyżej elementy umieszczone zostały w pliku nagłówkowym przedstawionym na listingu 4.8.

Listing 4.8. Plik nagłówkowy serwera echo

```

1  /*
2   * echo.h
3   *
4   * Created on: Oct 17, 2021
5   * Author: Kacper Wojciechowski
6   */
7
8  #ifndef INC_ECHO_H_
9  #define INC_ECHO_H_
10
11  #include "lwip/debug.h"

```

```

12 #include "lwip/stats.h"
13 #include "lwip/tcp.h"
14
15 /* Stany serwera echo */
16 enum tcp_state_enum {
17     TCP_STATE_NONE = 0,
18     TCP_STATE_ACCEPTED,
19     TCP_STATE_RECEIVED,
20     TCP_STATE_CLOSING
21 };
22
23 /* Info serwera echo */
24 struct echo_info {
25     uint8_t state;           // stan
26     uint8_t retries;        // licznik powtórzeń
27     struct tcp_pcb* pcb;    // wskaźnik PCB
28     struct pbuf* p;         // bufor pakietów
29 };
30
31 err_t init_echo(void);
32
33
34 #endif /* INC_ECHO_H_ */

```

Plik źródłowy połączony z powyższym plikiem nagłówkowym zawiera prototypy oraz implementacje funkcji zwrotnych oraz funkcji wysyłania i zakończenia połączenia. Zostały one przedstawione na listingu 4.9.

Listing 4.9. Fragment pliku źródłowego serwera echo

```

1  /*
2   * echo.c
3   *
4   * Created on: Oct 17, 2021
5   * Author: Kacper Wojciechowski
6   */
7
8  #include "echo.h"
9
10 static struct tcp_pcb* pcb_server; // echoserver pcb
11
12 /* Funkcje callback dla stosu LwIP */
13 static err_t app_callback_accepted(void* arg, struct tcp_pcb* pcb_new,
14                                     err_t err);
15 static err_t app_callback_received(void* arg, struct tcp_pcb* tpcb,
16                                     struct pbuf* p, err_t err);
17 static void app_callback_error(void* arg, err_t err);
18 static err_t app_callback_sent(void* arg, struct tcp_pcb* tpcb,
19                                uint16_t len);
20 static err_t app_callback_poll(void* arg, struct tcp_pcb* tpcb);
21
22 /* Funkcje użytkowe */
23 static void app_send_data(struct tcp_pcb* tpcb, struct echo_info* info);
24 static void app_close_connection(struct tcp_pcb* tpcb,
25                                 struct echo_info* info);

```

Uruchomienie serwera echo rozpoczyna się od inicjalizacji serwera za pomocą funkcji *init_echo()* wywołanej w funkcji *main()* przed pętlą główną. Funkcja ta tworzy nowy

obiekt serwera TCP, łączy go z wybranym portem oraz rejestruje funkcję zwrotną (ang. *callback*) zaakceptowania połączenia. Do komunikacji wybrano port 7. Listing 4.10. przedstawia kod funkcji *init_echo()*.

Listing 4.10. Funkcja inicjalizująca serwer TCP echo

```

1  /* Funkcja inicjalizująca serwer echo */
2  err_t init_echo()
3  {
4      err_t err;
5      pcb_server = tcp_new(); // alokacja pamięci pcb
6
7      if(pcb_server == NULL)
8      {
9          /* W przypadku braku pamięci, zwolnienie zaalokowanej pamięci
10           * i zwrócenie błędu */
11          memp_free(MEMP_TCP_PCB, pcb_server);
12          return ERR_MEM;
13      }
14
15      /* Powiązanie serwera z portem 7 */
16      err = tcp_bind(pcb_server, IP_ADDR_ANY, 7);
17      if (err != ERR_OK)
18      {
19          /* W przypadku błędu powiązania, zwolnienie pamięci i zwrócenie
20           * błędu */
21          memp_free(MEMP_TCP_PCB, pcb_server);
22          return err;
23      }
24
25      /* Nasłuchiwanie */
26      pcb_server = tcp_listen(pcb_server);
27      /* Zarejestrowanie funkcji zwrotnej zaakceptowania połączenia */
28      tcp_accept(pcb_server, app_callback_accepted);
29
30      return ERR_OK;
31  }
```

Funkcja *app_callback_accepted()* jest wywoływana w momencie akceptacji połączenia przez serwer. Ustawia ona priorytet utworzonego pcb, alokuje strukturę informacji o połączeniu *echo_info*, ustawia jej początkowe parametry oraz rejestruje funkcje zwrotne odbioru danych, błędu i pollingu. Kod funkcji został zamieszczony w listingu 4.11.

Listing 4.11. Funkcja zwrotna akceptacji połączenia

```

1  /* Funkcja zwrotna zaakceptowania połączenia */
2  static err_t app_callback_accepted(void* arg, struct tcp_pcb* pcb_new,
3                                     err_t err)
4  {
5      struct echo_info* info;
6
7      /* usunięcie ostrzeżeń */
8      (void)arg;
9      (void)err;
10
11     /* Ustawienie priorytetu nowego pcb */
```

```

12  tcp_setprio(pcb_new, TCP_PRIO_NORMAL);
13
14  /* alokacja struktury echo_info */
15  info = (struct echo_info*)mem_malloc(sizeof(struct echo_info));
16
17
18  if (info == NULL)
19  {
20      /* W przypadku braku pamięci, zamknięcie połączenia i zwrócenie
21       * błędu */
22      app_close_connection(pcb_new, info);
23      return ERR_MEM;
24  }
25
26  /* Ustawienie stanu połączenia na zaakceptowane */
27  info->state = TCP_STATE_ACCEPTED;
28  /* Zapisanie wskaźnika na pcb */
29  info->pcb = pcb_new;
30  /* Wyzerowanie licznika */
31  info->retries = 0;
32  /* Wyczyszczenie wskaźnika na bufor */
33  info->p = NULL;
34
35  /* Przesłanie struktury pcb_new jako argument */
36  tcp_arg(pcb_new, info);
37  /* Zarejestrowanie funkcji zwrotnej odbioru wiadomości */
38  tcp_recv(pcb_new, app_callback_received);
39  /* Zarejestrowanie funkcji zwrotnej błędu */
40  tcp_err(pcb_new, app_callback_error);
41  /* Zarejestrowanie funkcji zwrotnej nasłuchiwania */
42  tcp_poll(pcb_new, app_callback_poll, 0);
43
44  return ERR_OK;
45 }

```

Kolejną funkcję zwrotną stanowi *app_callback_received()*, wywoływana w trakcie otrzymania danych z połączonego urządzenia. Zajmuje się ona nadzorem buforów, wywołuje obsługę wysłania danych z powrotem do nadawcy w postaci echo oraz odpowiednio zmienia stan połączenia zapisany w strukturze *echo_info*. Rejestruje ona także funkcję zwrotną *app_callback_sent*. Jej kod jest zaprezentowany na listingu 4.12.

Listing 4.12. Funkcja zwrotna odbioru danych

```

1  /* Funkcja zwrotna odbioru wiadomości */
2  static err_t app_callback_received(void* arg, struct tcp_pcb* tpcb,
3                                     struct pbuf* p, err_t err)
4  {
5      /* Wskaźnik na strukturę echo_info */
6      struct echo_info* info;
7      /* Zmienna informacji o błędach */
8      err_t ret_err;
9
10     /* Sprawdzenie czy argument jest różny od NULL */
11     LWIP_ASSERT("arg!=_NULL", arg != NULL);
12     /* Zapisanie zrzutowanego wskaźnika na strukturę echo_info */
13     info = (struct echo_info*) arg;

```



```
14
15  /* Jeżeli funkcja została wywołana, ale nie ma danych */
16  if(p == NULL)
17  {
18      /* Ustawienie stanu połączenia na zamykane */
19      info->state = TCP_STATE_CLOSING;
20      /* Jeżeli bufor struktury jest NULL */
21      if(info->p == NULL)
22      {
23          /* Zamknięcie połączenia */
24          app_close_connection(tpcb, info);
25      }
26      /* Jeżeli w buforze struktury są dane do przesłania */
27      else
28      {
29          /* Zarejestrowanie funkcji zwrotnej wysyłania danych */
30          tcp_sent(tpcb, app_callback_sent);
31          /* Przesłanie danych */
32          app_send_data(tpcb, info);
33      }
34      ret_err = ERR_OK;
35  }
36  /* Jeśli wystąpił błąd przy odbiorze */
37  else if(err != ERR_OK)
38  {
39      /* Jeżeli bufor nie jest pusty */
40      if(p != NULL)
41      {
42          /* Wyczyszczenie buforu struktury */
43          info->p = NULL;
44          /* Zwolnienie buforu */
45          pbuf_free(p);
46      }
47      /* Zapisanie informacji o błędzie */
48      ret_err = err;
49  }
50  /* Odbiór pierwszych danych */
51  else if (info->state == TCP_STATE_ACCEPTED)
52  {
53      /* Zmiana statusu na odebrano */
54      info->state = TCP_STATE_RECEIVED;
55      /* Zapisanie buforu w strukturze */
56      info->p = p;
57      /* Zarejestrowanie funkcji zwrotnej wysyłania danych */
58      tcp_sent(tpcb, app_callback_sent);
59      /* Przesłanie danych */
60      app_send_data(tpcb, info);
61      ret_err = ERR_OK;
62  }
63  /* Odbiór kolejnych danych */
64  else if (info->state == TCP_STATE_RECEIVED)
65  {
66      /* Jeżeli nie ma danych do wysłania */
67      if(info->p == NULL)
68      {
69          /* Zapisanie buforu w strukturze */
70          info->p = p;
71          /* Wysłanie danych */
```

```

72     app_send_data(tpcb, info);
73 }
74 /* Jeżeli są dane do przesłania, a bufor struktury nie jest
75  * pusty */
76 else
77 {
78     /* Utworzenie wskaźnika na bufor struktury */
79     struct pbuf* ptr = info->p;
80     /* Dołączenie do buforu struktury buforu otrzymanych
81      * danych */
82     pbuf_chain(ptr, p);
83 }
84 ret_err = ERR_OK;
85 }
86 /* Odbiór danych w trakcie zamykania */
87 else if(info->state == TCP_STATE_CLOSING)
88 {
89     /* Rekomendowanie rozmiaru okna */
90     tcp_recved(tpcb, p->tot_len);
91     /* Wyczyszczenie i zwolnienie buforów */
92     info->p = NULL;
93     pbuf_free(p);
94     ret_err = ERR_OK;
95 }
96 /* Odbiór zakończony */
97 else
98 {
99     /* Rekomendowanie rozmiaru okna */
100    tcp_recved(tpcb, p->tot_len);
101    /* Wyczyszczenie i zwolnienie buforów */
102    info->p = NULL;
103    pbuf_free(p);
104    ret_err = ERR_OK;
105 }
106 return ret_err;
107 }

```

Funkcja *app_callback_sent()* odpowiada za nadzór sytuacji po wykonaniu transmisji. W przypadku wystąpienia dalszych danych do przesłania, inicjalizuje ona kolejną transmisję. Podobnie działa funkcja zwrotna *app_callback_poll()* Listingi 4.13 i 4.14 przedstawiają kod wspomnianych funkcji.

Listing 4.13. Funkcja zwrotna transmisji danych

```

1  /* Funkcja zwrotna wysyłania */
2  static err_t app_callback_sent(void* arg, struct tcp_pcb* tpcb, uint16_t len)
3  {
4      /* Wskaźnik na strukturę */
5      struct echo_info* info;
6      /* Uciszenie ostrzeżeń */
7      (void)len;
8
9      /* Zapisanie zrzutowanego wskaźnika na strukturę */
10     info = (struct echo_info*) arg;
11     /* Wyzerowanie licznika ponownych prób */
12     info->retries = 0;
13
14     /* Jeżeli są dane do wysłania */

```

```

15  if(info->p != NULL)
16  {
17      /* Zarejestrowanie funkcji zwrotnej wysyłania danych */
18      tcp_sent(tpcb, app_callback_sent);
19      /* Przesłanie danych */
20      app_send_data(tpcb, info);
21  }
22  /* Jeżeli nie ma danych do wysłania */
23  else
24  {
25      /* Jeżeli stan połączenia jest jako zamykane */
26      if(info->state == TCP_STATE_CLOSING)
27      {
28          /* Zamknięcie połączenia */
29          app_close_connection(tpcb, info);
30      }
31  }
32  return ERR_OK;
33 }

```

Listing 4.14. Funkcja zwrotna nasłuchu

```

1  /* Funkcja zwrotna nasłuchu */
2  static err_t app_callback_poll(void* arg, struct tcp_pcb* tpcb)
3  {
4      /* Wskaźnik na strukturę */
5      struct echo_info* info;
6      /* Zapisanie zrzutowanego wskaźnika na strukturę */
7      info = (struct echo_info*) arg;
8      /* Jeżeli nie ma struktury */
9      if(info == NULL)
10     {
11         /* Przerwanie połączenia i zwrócenie błędu */
12         tcp_abort(tpcb);
13         return ERR_ABRT;
14     }
15     /* Jeżeli są dane do przesłania */
16     if(info->p != NULL)
17     {
18         /* Zarejestrowanie funkcji zwrotnej wysyłania danych */
19         tcp_sent(tpcb, app_callback_sent);
20         /* Przesłanie danych */
21         app_send_data(tpcb, info);
22     }
23     /* Jeżeli nie ma danych do przesłania */
24     else
25     {
26         /* Jeżeli status połączenia jest jako zamykane */
27         if(info->state == TCP_STATE_CLOSING)
28         {
29             /* Zamknięcie połączenia */
30             app_close_connection(tpcb, info);
31         }
32     }
33     return ERR_OK;
34 }

```

Ostatnią z funkcji zwrotnych jest *app_callback_error()* wywoływana w momencie

wystąpienia błędu podczas transmisji. Odpowiada ona za sygnalizację błędu za pomocą niebieskiej diody LED oraz zwolnienie zaalokowanej struktury *echo_info*. Jej kod został ukazany na listingu 4.15.

Listing 4.15. Funkcja zwrotna błędu

```
1  /* Funkcja zwrotna błędu */
2  static void app_callback_error(void* arg, err_t err)
3  {
4      /* Wskaźnik na strukturę echo_info */
5      struct echo_info* info;
6      /* Uciszenie ostrzeżeń */
7      (void)err;
8
9      /* Przypisanie zrzuconego wskaźnika na strukturę */
10     info = (struct echo_info*) arg;
11     /* Jeżeli zapisano strukturę echo_info */
12     if (info != NULL)
13     {
14         /* Zwolnienie zaalokowanej pamięci */
15         mem_free(info);
16     }
17     /* Włączenie niebieskiej diody LED jako sygnalizację błędu */
18     HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_SET);
19 }
```

Kluczowym elementem serwera echo jest funkcja realizująca transmisję otrzymanych danych z powrotem do nadawcy. Jest to zadanie funkcji *app_send_data()*, wykorzystującej do transmisji funkcję stosu LwIP *tcp_write()*. Obsługuje ona transmisję ciągu pakietów oraz nadzoruje poprawność przeprowadzenia transmisji, w razie potrzeby wykonując retransmisję. Listing 4.16 przedstawia kod funkcji.

Listing 4.16. Funkcja transmisji danych

```
1  /* Funkcja przesyłu danych */
2  static void app_send_data(struct tcp_pcb* tpcb, struct echo_info* info)
3  {
4      /* Bufor na dane do wysłania */
5      struct pbuf* ptr;
6      err_t wr_err = ERR_OK;
7
8      /* Dopóki nie napotkano błędu, są dane do wysłania, i długość danych
9       * jest mniejsza niż długość buforu wysyłkowego */
10     while((wr_err == ERR_OK) && (info->p != NULL)
11           && (info->p->len <= tcp_sndbuf(tpcb)))
12     {
13         /* Przepisanie wskaźnika na bufor */
14         ptr = info->p;
15         /* Przesłanie danych */
16         wr_err = tcp_write(tpcb, ptr->payload, ptr->len,
17                           TCP_WRITE_FLAG_COPY);
18
19         /* Jeżeli dane zostały przesłane prawidłowo */
20         if(wr_err == ERR_OK)
21         {
22             /* Zmienna na długość bufora */
23             uint16_t plen;
```

```

24     uint8_t freed;
25
26     /* Zapisanie długości bufora */
27     plen = ptr->len;
28     /* Przetawienie bufora na następny pakiet */
29     info->p = ptr->next;
30
31     /* Jest do przesłania bufor łańcuchowy */
32     if(info->p != NULL)
33     {
34         /* Zwiększenie wskaźnika referencyjnego */
35         pbuf_ref(info->p);
36     }
37
38     do
39     {
40         /* Zwolnienie starego bufora */
41         freed = pbuf_free(ptr);
42     }
43     while(freed == 0);
44
45     /* Rekomendowanie rozmiaru okna */
46     tcp_recved(tpcb, plen);
47 }
48 /* Jeżeli nie udało się przesłać danych */
49 else
50 {
51     /* Odzyskanie wskaźnika na bufor */
52     info->p = ptr;
53     /* Zwiększenie licznika powtórzeń transmisji */
54     info->retries++;
55 }
56 }
57 }

```

Po zakończeniu pracy z serwerem echo konieczne jest zamknięcie połączenia, za co odpowiada funkcja `app_close_connection()`. Odrejestrowuje ona ustawione wcześniej funkcje zwrotne, zamyka połączenie oraz zwalnia alokację zasobów. Kod funkcji przedstawiony jest na listingu 4.17.

Listing 4.17. Funkcja zamknięcia połączenia

```

1  /* Funkcja zamykająca połączenie */
2  static void app_close_connection(struct tcp_pcb* tpcb, struct echo_info* info)
3  {
4      /* Wyczyszczenie funkcji zwrotnych */
5      tcp_arg(tpcb, NULL);
6      tcp_sent(tpcb, NULL);
7      tcp_recv(tpcb, NULL);
8      tcp_err(tpcb, NULL);
9      tcp_poll(tpcb, NULL, 0);
10
11     /* Jeżeli zapisano strukturę */
12     if(info != NULL)
13     {
14         /* Zwolnienie zaalokowanej pamięci */
15         mem_free(info);
16     }

```

```

17
18     /* Zamknięcie połączenia */
19     tcp_close(tpcb);
20 }

```

4.4.2. Interfejs bezprzewodowy Wi-Fi

W celu rozdzielania implementacji interfejsu Ethernet i interfejsu bezprzewodowego, w środowisku STM32CubeIDE utworzony został nowy projekt.

Implementacja interfejsu bezprzewodowego Wi-Fi została rozpoczęta od przygotowania odpowiedniej konfiguracji mikrokontrolera STM32-F429ZI. Wykorzystany moduł Wi-Fi komunikuje się z mikrokontrolerem za pomocą interfejsu UART, używając przy tym komend ze standardu AT. W związku z tym, konfiguracja rozpoczęła się od aktywowania dwóch interfejsów typu UART:

- USART3 - port szeregowy w celu komunikacji z komputerem nadzorującym,
- USART6 - port szeregowy służący do komunikacji z modulem Wi-Fi.

Obydwa porty zostały skonfigurowane do pracy z częstotliwością 115200 bitów na sekundę, 8-bitową długością słowa, brakiem kontroli parzystości i pojedynczym bitem stopu. Konfigurację obu portów przedstawiono na rysunku 4.13.

▼ Basic Parameters	
Baud Rate	115200 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1
▼ Advanced Parameters	
Data Direction	Receive and Transmit
Over Sampling	16 Samples

Figure 4.13. Konfiguracja interfejsów USART3 oraz USART6

Kolejny krok stanowiło ustawienie w zakładce RCC sekcji System Core zegara HSE w tryb BYPASS, podobnie jak w przypadku interfejsu Ethernet, aby wykorzystać do taktowania mikrokontrolera sygnał zegarowy generowany przez moduł ST-Link.

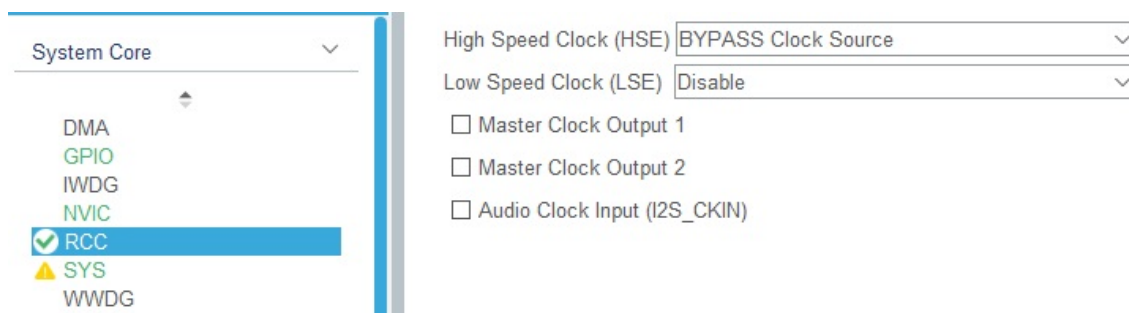


Figure 4.14. Ustawienie zegara HSE

W zakładce *Clock Configuration* częstotliwość taktowania zegara systemowego została ustawiona z pomocą konfiguratora zegarów na 180 MHz, co przedstawiono na rysunku 4.15.

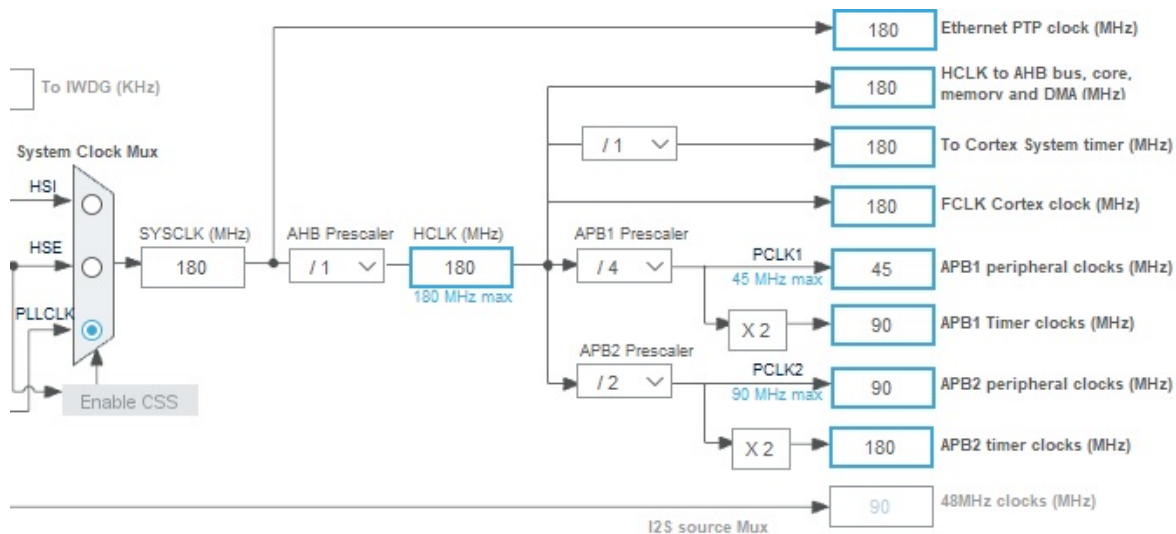


Figure 4.15. Częstotliwości końcowe w menadżerze zegarów

Ostatnim krokiem konfiguracji była konfiguracja jednego z portów GPIO w tryb wyjścia, w celu sterowania linią *enable* modułu Wi-Fi. W celu uaktywnienia modułu i możliwości pracy z nim, konieczne było podanie stanu wysokiego na linię *enable*. Wykorzystano do tego linię PB3, zachowując jej domyślne parametry, których podgląd i modyfikacja dostępna jest w zakładce GPIO sekcji *System Core*. Zostały one przedstawione na rysunku 4.16 i 4.17.

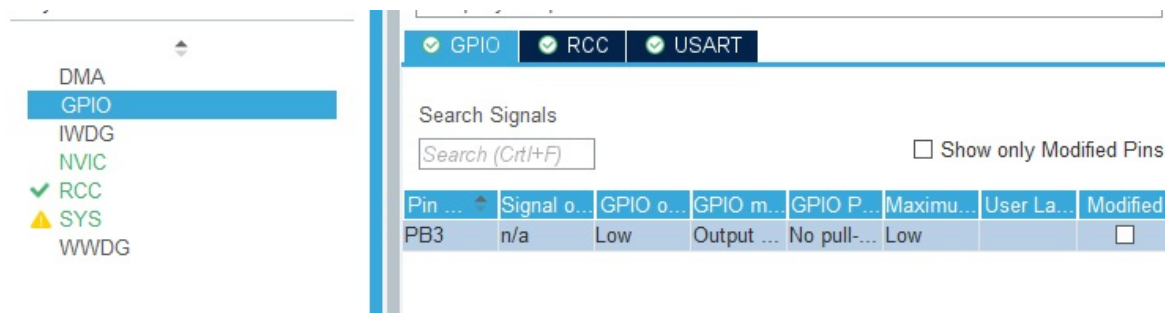


Figure 4.16. Lista skonfigurowanych linii GPIO

Moduł ESP8266 posiada wbudowaną implementację stosu TCP/IP, w związku z czym aktywowanie stosu LwIP nie było konieczne. Na podstawie uzyskanej konfiguracji wygenerowany został kod źródłowy, który następnie został rozbudowany o polecenia konfiguracyjne modułu Wi-Fi oraz funkcje realizujące rolę serwera echo. Listing 4.18 przedstawia kod funkcji *main()* i elementy globalne.

PB3 Configuration :

GPIO output level	Low
GPIO mode	Output Push Pull
GPIO Pull-up/Pull-down	No pull-up and no pull-down
Maximum output speed	Low
User Label	

Figure 4.17. Podgląd parametrów linii PB3

Listing 4.18. Funkcja *main()* i elementy globalne

```

1  /* Makro określające ilość komend konfiguracyjnych */
2  #define COMMANDS 12
3
4  /* Bufor odbieranych komunikatów od modułu */
5  char data_r[1000];
6  /* Bufor odebranej wiadomości */
7  char message[100];
8
9  /* Flaga gotowości do wysłania wiadomości echo */
10 int ready_to_send = 0;
11
12 /* Tablica komend konfiguracyjnych modułu */
13 char* config[] = {
14     "AT+RST\r\n",           /* Zresetuj moduł */
15     "ATE0\r\n",             /* Wyłącz echo komend */
16     "AT\r\n",               /* Test komunikacji */
17     "AT+GMR\r\n",           /* Wersja oprogramowania */
18     "AT+CWMODE=1\r\n",      /* Tryb pracy jako klient */
19     "AT+CIPMODE=0\r\n",     /* Wybranie formatu odbieranych
20                             * wiadomości */
21     "AT+CIPMUX=1\r\n",      /* Ustawienie obsługi wielu
22                             * połączeń */
23     "AT+CIPSERVER=1,7\r\n", /* Włączenie serwera i
24                             * ustawienie portu 7 */
25     "AT+CWMODE=?\r\n",      /* Sprawdzenie trybu
26                             * pracy */
27     "AT+CWJAP=\"marx\", \"*****\" \r\n", /* Połączenie z siecią
28                             * Wi-Fi */
29     "AT+CIPSTA?\r\n",       /* Sprawdzenie statusu
30                             * połączenia z Wi-Fi */
31     "AT+CIFSR\r\n"          /* Wyświetlenie adresu
32                             * IP serwera */
33 };
34
35 int main(void)
36 {
37     /* USER CODE BEGIN 1 */

```



```
38
39  /* USER CODE END 1 */
40
41  /* MCU Configuration-----*/
42
43  /* Reset of all peripherals, Initializes the Flash interface and
44   * the SysTick. */
45  HAL_Init();
46
47  /* USER CODE BEGIN Init */
48
49  /* USER CODE END Init */
50
51  /* Configure the system clock */
52  SystemClock_Config();
53
54  /* USER CODE BEGIN SysInit */
55
56  /* USER CODE END SysInit */
57
58  /* Initialize all configured peripherals */
59  MX_GPIO_Init();
60  MX_USART3_UART_Init();
61  MX_USART6_UART_Init();
62  /* USER CODE BEGIN 2 */
63  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_3, GPIO_PIN_SET);
64
65  /* USER CODE END 2 */
66
67  /* Infinite loop */
68  /* USER CODE BEGIN WHILE */
69
70  /* Pętla konfiguracji modułu przesyłająca kolejne komendy konfiguracyjne */
71  for(i = 0; i < COMMANDS; i++)
72  {
73      /* Przesłanie komendy konfiguracyjnej */
74      HAL_UART_Transmit(&huart6, (uint8_t*)(config[i]), strlen(config[i]), 10);
75      /* Wyczyszczenie bufora odbiorczego */
76      clr_buffer(data_r, 1000);
77      /* Odbiór komunikatu zwrotnego */
78      HAL_UART_Receive(&huart6, (uint8_t*)data_r, sizeof(data_r), 100);
79
80      /* Odczekanie sekundy po komendzie restartu modułu */
81      if(i == 0)
82      {
83          HAL_Delay(1000);
84      }
85
86      /* Pętla oczekiwania na otrzymanie adresu z DHCP routera */
87      while(is_busy())
88      {
89          /* Odczekanie sekundy */
90          HAL_Delay(1000);
91          /* Wysłanie komendy sprawdzającej status otrzymanego adresu IP,
92           * maski oraz bramy */
93          HAL_UART_Transmit(&huart6, (uint8_t*)(config[i]), strlen(config[i]),
94                           10);
95          /* Wyczyszczenie bufora odbiorczego */
```

```
96     clr_buffer(data_r, 1000);
97     /* Otrzymanie komunikatu zwrotnego */
98     HAL_UART_Receive(&huart6, (uint8_t*)data_r, sizeof(data_r),
99                     100);
100 }
101 /* Przesłanie komunikatu zwrotnego poprzez port szeregowy do
102  * komputera nadzorującego */
103 HAL_UART_Transmit(&huart3, (unsigned char*)data_r, sizeof(data_r),
104                  10);
105 HAL_UART_Transmit(&huart3, (unsigned char*)"\r\n\n", sizeof("\r\n\n"),
106                  10);
107
108 }
109
110 while (1)
111 {
112     /* USER CODE END WHILE */
113     /* Wyczyszczenie bufora odbiorczego */
114     clr_buffer(data_r, 1000);
115     /* Pobór komunikatu z modułu */
116     HAL_UART_Receive(&huart6, (uint8_t*)data_r, sizeof(data_r), 100);
117     /* Jeżeli ustawiona jest flaga gotowości do wysłania wiadomości echo */
118     if(ready_to_send)
119     {
120         /* Wysłanie wiadomości echo */
121         HAL_UART_Transmit(&huart6, (uint8_t*)message, sizeof(message), 10);
122         /* Reset flagi gotowości do wysłania wiadomości echo */
123         ready_to_send = 0;
124     }
125     /* W przeciwnym wypadku, jeżeli otrzymany komunikat nie jest pustym
126      * komunikatem */
127     else if (data_r[0] != 0)
128     {
129         /* Przygotowanie do wysłania wiadomości echo */
130         prepare_echo();
131         /* Wysłanie otrzymanego komunikatu za pomocą portu szeregowego */
132         HAL_UART_Transmit(&huart3, (unsigned char*)data_r, sizeof(data_r),
133                          10);
134         HAL_UART_Transmit(&huart3, (unsigned char*)"\r\n\n",
135                          sizeof("\r\n\n"), 10);
136     }
137
138     /* USER CODE BEGIN 3 */
139 }
140 /* USER CODE END 3 */
141 }
```

Jako elementy globalne zadeklarowane zostały:

- bufor znakowy odbioru informacji zwrotnych z modułu Wi-Fi,
- bufor znakowy sformatowanej wiadomości do odesłania do nadawcy,
- makro określające ilość komend konfiguracyjnych modułu Wi-Fi,
- tablica komend konfiguracyjnych modułu Wi-Fi.

Działanie każdej z komend konfiguracyjnych opisane zostało za pomocą komentarzy w listingu 4.18. W celu ochrony prywatności hasło zamieszczone na listingu dla komendy *AT+CWJAP* zostało zamienione na znaki gwiazdki.

Funkcja *main()* składa się z kilku kluczowych sekcji. Pierwszą z nich jest wygenerowana sekcja konfiguracji poszczególnych interfejsów i elementów mikrokontrolera, takich jak GPIO, zegar systemowy oraz interfejsy USART. Sekcja ta została rozszerzona o ustawienie linii PB3 podłączonej do wejścia *enable* modułu Wi-Fi w stan wysoki.

Drugą sekcją jest pętla konfiguracyjna. Jej zadaniem jest dokonanie resetu oraz konfiguracji modułu Wi-Fi w sposób pozwalający na nawiązanie połączenia i dwustronną transmisję wiadomości. Są to kolejne polecenia:

- *AT+RST* - restart modułu;
- *ATE0* - wyłączenie echa wysłanych komend konfiguracyjnych;
- *AT* - test komunikacji z modułem;
- *AT+GMR* - uzyskanie informacji o wersji oprogramowania modułu;
- *AT+CWMODE=1* - ustawienie trybu pracy jako klient;
- *AT+CIPMODE=0* - ustawienie formatu odbieranych wiadomości jako "+IPD,<kanał>,<liczba bajtów>,<dane>";
- *AT+CIPMUX=1* - ustawienie możliwości obsługi wielu połączeń;
- *AT+CIPSERVER=1,7* - włączenie funkcjonalności serwera na porcie 7;
- *AT+CWMODE=?* - sprawdzenie ustawionego trybu pracy;
- *AT+CWJAP=<SSID>,<hasło>* - połączenie z siecią Wi-Fi;
- *AT+CIPSTA?* - sprawdzenie statusu połączenia;
- *AT+CIFSR* - pobranie adresu IP serwera;

Po przesłaniu polecenia restartu modułu, mikrokontroler odczeka 1 sekundę przed przesłaniem kolejnych poleceń. Na transmisję każdego z poleceń składa się przesłanie polecenia do modułu, wyczyszczenie bufora odbiorczego za pomocą funkcji *clr_buffer()*, ukazanej na listingu 4.19, i odebranie wiadomości zwrotnej z modułu. Po poleceniu *AT+CWJAP* występuje oczekiwanie na otrzymanie adresu DHCP poprzez wysyłanie w interwałach 1-sekundowych zapytania o status połączenia za pomocą funkcji *is_busy()*, której kod przedstawia listing 4.20. Tak długo, jak status informuje o zajętości modułu, następuje oczekiwanie i ponowne zapytanie. Informacje zwrotne odebrane z modułu przesyłane są do komputera nadzorującego za pomocą portu szeregowego.

Listing 4.19. Funkcja czyszcząca bufor odbiorczy

```
1  /* Funkcja czyszczenia buforu odbiorczego */
2  static void clr_buffer(char* buff, int size)
3  {
4      int i;
5      /* Nadpisanie całego buforu wartością 0 */
6      for(i = 0; i < size; i++)
7      {
8          buff[i] = 0;
9      }
10 }
```

Listing 4.20. Funkcja sprawdzania stanu zajętości modułu Wi-Fi

```
1  /* Funkcja oczekująca na uzyskanie adresu z DHCP routera */
2  static int is_busy(void)
3  {
4      /* Zmienna na wartość zwracaną */
5      int ret = 0;
6      int i;
7      /* Wyszukaj frazę "busy" w otrzymanym z modułu komunikacie */
8      for(i = 0; i < sizeof(data_r); i++)
9      {
10         if(data_r[i] == 'b' && data_r[i+1] == 'u' && data_r[i+2] == 's'
11            && data_r[i+3] == 'y')
12         {
13             /* Jeśli znaleziono frazę, zwróć informację że moduł jest
14              * zajęty oczekiwaniem na DHCP */
15             ret = 1;
16             break;
17         }
18     }
19     /* Zwróć informację o zajętości modułu */
20     return ret;
21 }
```

Trzecią sekcją funkcji *main()* jest pętla operacyjna, realizująca funkcjonalność serwera echo. Wewnątrz niej czyszczony jest bufor odbiorczy i odbierane są dane wysyłane z modułu przy wystąpieniu zdarzenia, takiego jak nawiązanie połączenia lub odbiór danych. W przypadku odebrania niepustej wiadomości zwrotnej, wykonywane jest przygotowanie do funkcjonalności echo za pomocą funkcji *prepare_echo()*, której kod znajduje się na listingu 4.21.

Listing 4.21. Funkcja przygotowująca funkcjonalność echo

```
1  /* Funkcja przygotowująca mikrokontroler do wysłania wiadomości echo */
2  static void prepare_echo(void)
3  {
4      /* Bufor na długość wiadomości - obsługa wiadomości do 99 znaków */
5      char byte_count[3];
6      /* Bufor na komendę umożliwiającą wysłanie wiadomości */
7      char command[20];
8      int i;
9
10     /* Sprawdzenie czy otrzymany z modułu komunikat mówi o otrzymaniu
11      * przez moduł wiadomości */
```

```

12  /* Rozpoczęcie od indeksu 2 wynika z offsetu obecnego na początku
13  * wiadomości */
14  if(data_r[2] == '+' && data_r[3] == 'I' && data_r[4] == 'P'
15     && data_r[5] == 'D')
16  {
17     /* Przypisanie indeksu pierwszej cyfry ilości znaków otrzymanej
18     * wiadomości */
19     i = 9;
20     /* Wyciągnięcie ilości znaków otrzymanej wiadomości */
21     while(data_r[i] != ':')
22     {
23         byte_count[i-9] = data_r[i];
24         i++;
25     }
26     /* Zakończenie ciągu znakowego */
27     byte_count[i-9] = '\0';
28     /* Ustawienie indeksu pierwszej litery otrzymanej wiadomości */
29     i++;
30     /* Wyciągnięcie otrzymanej wiadomości */
31     while(data_r[i] != 0)
32     {
33         message[i-11] = data_r[i];
34         i++;
35     }
36     /* Zakończenie wiadomości */
37     message[i++] = '\r';
38     message[i++] = '\n';
39     message[i] = '\0';
40     /* Sformatowanie komendy umożliwiającej wysłanie wiadomości */
41     sprintf(command, "AT+CIPSEND=0,%d\r\n", atoi(byte_count));
42     /* Wysłanie komendy do modułu */
43     HAL_UART_Transmit(&huart6, (uint8_t*)(command), strlen(command), 10);
44     /* Ustawienie flagi gotowości do wysłania wiadomości echo */
45     ready_to_send=1;
46 }
47 }

```

Jeżeli otrzymana wiadomość zwrotna z modułu rozpoczyna się od sekwencji "+IPD", wyciągana jest z niej ilość otrzymanych bajtów oraz jej treść w celu odpowiedniego sformatowania i przygotowania do wysłania. Gotowość do wysłania oznaczana jest za pomocą flagi *ready_to_send*. W przypadku wyciągnięcia wiadomości, wysyłane jest polecenie przesłania danych, a następnie w trzeciej sekcji funkcji *main()* podawana jest do transmisji wiadomość uzyskana od nadawcy. Po wykonaniu transmisji echo, flaga *ready_to_send* jest resetowana.

4.5. Testy rozwiązania

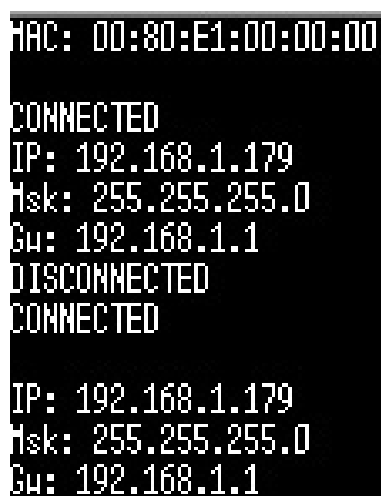
4.5.1. Interfejs Ethernet

Podczas przeprowadzania testów sprawdzono następujące funkcjonalności:

- komunikacja z komputerem nazdorującym za pomocą portu szeregowego, otrzymanie informacji o adresie MAC oraz stanach połączenia;

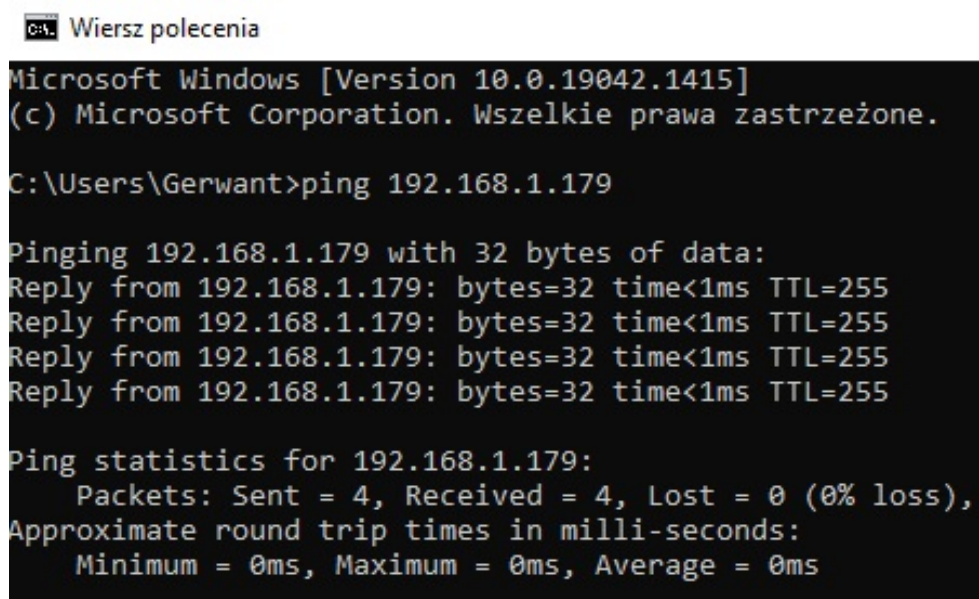
- przesłanie polecenia ping za pomocą wiersza poleceń systemu Windows;
- połączenie i transmisja dwustronna za pomocą programu Hercules SETUP Utility;

Rysunki 4.18, 4.19 oraz 4.20 przedstawiają rezultaty przeprowadzonych testów. Pierwszym z działań było uzyskanie informacji o adresie IP modułu za pomocą interfejsu szeregowego USART, co przedstawiono na rysunku 4.18. Następnie możliwe było sprawdzenie poprawności komunikacji za pomocą polecenia *ping*, co pokazano na rysunku 4.19 oraz poprawności działania funkcjonalności serwera echo za pomocą programu Hercules SETUP Utility ukazanego na rys. 4.20. Wszystkie testy zakończyły się pomyślnie, weryfikując poprawne działanie funkcjonalności interfejsu sieciowego Ethernet.



```
MAC: 00:80:E1:00:00:00
CONNECTED
IP: 192.168.1.179
Hsk: 255.255.255.0
Gu: 192.168.1.1
DISCONNECTED
CONNECTED
IP: 192.168.1.179
Hsk: 255.255.255.0
Gu: 192.168.1.1
```

Figure 4.18. Komunikacja za pomocą portu szeregowego



```
Wiersz polecenia
Microsoft Windows [Version 10.0.19042.1415]
(c) Microsoft Corporation. Wszelkie prawa zastrzeżone.

C:\Users\Gerwant>ping 192.168.1.179

Pinging 192.168.1.179 with 32 bytes of data:
Reply from 192.168.1.179: bytes=32 time<1ms TTL=255
Reply from 192.168.1.179: bytes=32 time<1ms TTL=255
Reply from 192.168.1.179: bytes=32 time<1ms TTL=255
Reply from 192.168.1.179: bytes=32 time<1ms TTL=255

Ping statistics for 192.168.1.179:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Figure 4.19. Sprawdzenie połączenia za pomocą polecenia ping

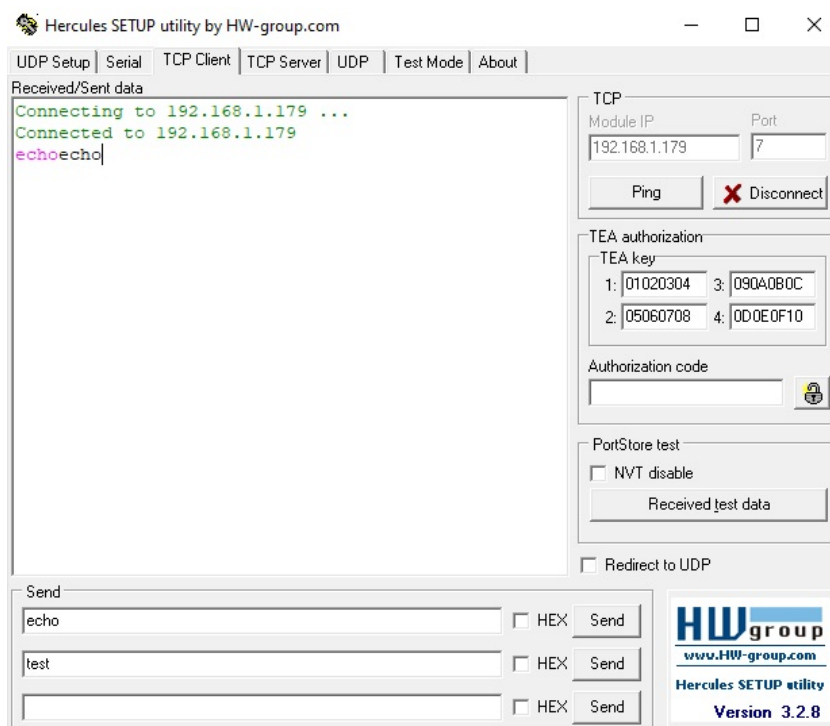


Figure 4.20. Komunikacja dwustronna z wykorzystaniem programu Hercules SETUP Utility

4.5.2. Interfejs bezprzewodowy Wi-Fi

Rysunek 4.21 przedstawia zdjęcie przygotowanego układu. Podobnie jak w przypadku interfejsu Ethernet, przeprowadzony został ten sam zestaw testów. W wyniku restartu urządzenia, na rysunku 4.22 zauważyć można nieczytelną wiadomość spowodowaną inną prędkością transmisji w trakcie pracy programu *bootloader* modułu. Podobnie jak w przypadku testów interfejsu Ethernet, najpierw uzyskano adres IP modułu za pomocą interfejsu USART, co pokazano na rys. 4.22 oraz 4.23. Następnie przeprowadzono test komunikacji za pomocą polecenia *ping*, co przedstawiono na rys. 4.24 oraz test funkcjonalności serwera echo za pomocą programu Hercules SETUP Utility pokazany na rys. 4.25. Wszystkie testy zakończyły się powodzeniem.

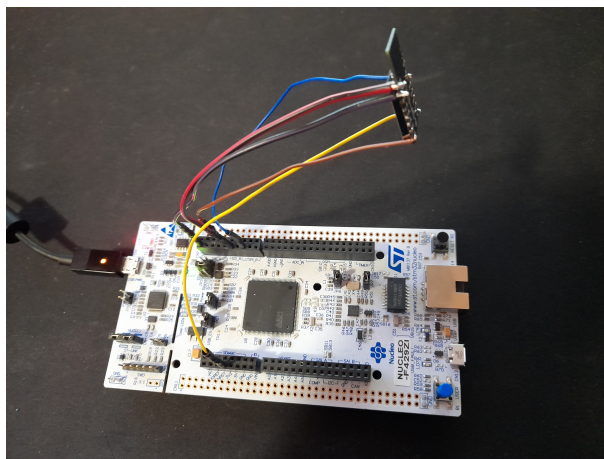


Figure 4.21. Zdjęcie przygotowanego układu


```

AT+RST
OK
i0f0p0ng0r0x0bb
1
lg0r0i0ft0p0e0i0i
1'i0ft0p0e0g0

ATE0
OK

OK

AT version:1.2.0.0(Jul 1 2016 20:04:45)
SDK version:1.5.4.1(39cb9a32)
Ai-Thinker Technology Co. Ltd.
Dec 2 2016 14:21:16

OK

OK

OK

```

Figure 4.22. Komunikacja z komputerem nadzorującym cz.1

```

OK

+CHMODE:(1-3)

OK

+CIPSTA:ip:"192.168.1.51"
+CIPSTA:gateway:"192.168.1.1"
+CIPSTA:netmask:"255.255.255.0"

OK

+CIFSR:STAIP,"192.168.1.51"
+CIFSR:STAMAC,"e8:db:84:84:06:cd"

OK

0,CONNECT

+IPD,0,4:echo

busy s...

Recv 4 bytes

SEND OK

0,CLOSED

```

Figure 4.23. Komunikacja z komputerem nadzorującym cz. 2

```

C:\Users\Gerwant>ping 192.168.1.51

Pinging 192.168.1.51 with 32 bytes of data:
Reply from 192.168.1.51: bytes=32 time=1ms TTL=128
Reply from 192.168.1.51: bytes=32 time=8ms TTL=128
Reply from 192.168.1.51: bytes=32 time=1ms TTL=128
Reply from 192.168.1.51: bytes=32 time=1ms TTL=128

Ping statistics for 192.168.1.51:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1ms, Maximum = 8ms, Average = 2ms

```

Figure 4.24. Sprawdzenie połączenia za pomocą polecenia ping

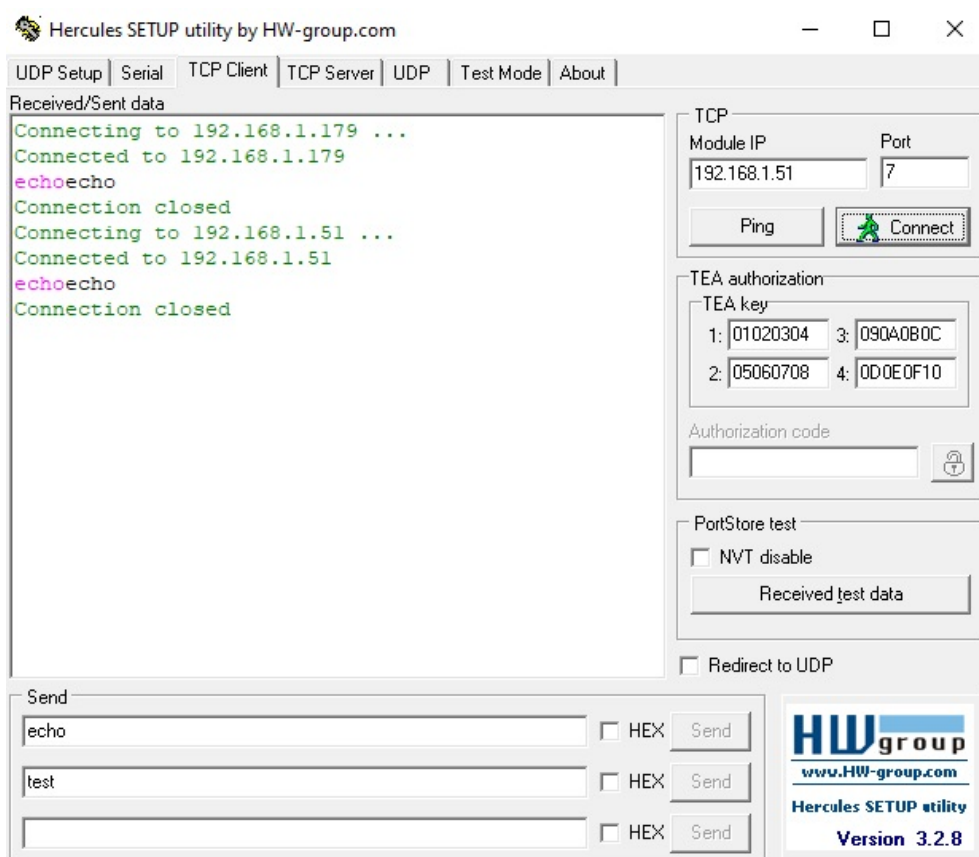


Figure 4.25. Komunikacja dwustronna z wykorzystaniem programu Hercules SETUP Utility

Rozdział 5

Dyskusja rezultatów i wnioski końcowe

W wyniku analizy istniejących rozwiązań implementacyjnych interfejsów sieciowych oraz analizy struktury stosu TCP/IP podjęte zostały decyzje o wyborze platform sprzętowych do implementacji modułu IoT w postaci serwera echo z obsługą połączeń sieciowych w części praktycznej niniejszej pracy.

Podczas realizacji niniejszej pracy wyciągnięte zostały następujące uwagi i wnioski:

- Istnieje wiele dostępnych na rynku rozwiązań implementacyjnych interfejsów sieciowych, w tym zarówno sieci przewodowych jak i bezprzewodowych;
- Część platform sprzętowych posiada gotowe implementacje części sprzętowej interfejsów sieciowych, jak np. wlutowane złącze 8P8C lub moduł Wi-Fi umieszczony na pokładzie płytki PCB;
- Istnieje kilka implementacji stosu TCP/IP różniących się od siebie kwestiami takimi jak oferowane funkcjonalności oraz zużycie pamięci i czasu przetwarzania. Przykładami implementacji stosu TCP/IP o otwartej licencji są stosy LwIP oraz uIP.
- Mimo zastosowania tego samej częstotliwości zegara systemowego, na podstawie obserwacji szybkości pojawiania się komunikatów w programie Hercules SETUP Utility i terminalu portu szeregowego, oraz na podstawie czasów zmierzonych przez polecenie ping, interfejs Wi-Fi charakteryzował się większym opóźnieniem przy przesyłaniu wiadomości echo niż interfejs Ethernet. W przypadku funkcjonalności echo oraz komunikacji za pomocą portu szeregowego prawdopodobną przyczyną jest konieczność komunikacji z układem mikrokontrolera za pomocą interfejsu UART oraz czas przetwarzania przez niego danych.

Bibliografia

- [1] *Internet of Things Global Standard Initiative*.
<https://www.itu.int/en/ITU-T/gsi/iot/Pages/default.aspx>, July 2021.
- [2] *Model OSI/ISO*. https://kisi.pcz.pl/Robert_Nowicki/psk-d/slides/030%20Model%20SI.pdf, Aug. 2021.
- [3] *Difference Between TCP/IP and OSI Model*.
<https://techdifferences.com/difference-between-tcp-ip-and-osi-model.html>, Sept. 2021.
- [4] *Protokoły TCP/IP*.
<http://www.crypto-it.net/pl/teoria/protokoly-tcp-ip.html>, Sept. 2021.
- [5] *HTTP*. <https://developer.mozilla.org/pl/docs/Web/HTTP>, Sept. 2021.
- [6] *Protokół SMTP (Simple Mail Transfer Protocol)*.
<https://www.ibm.com/docs/pl/i/7.3?topic=information-smtp>, Sept. 2021.
- [7] *Różnice pomiędzy protokołami POP3 i IMAP*.
https://www.uci.umk.pl/index.php/R%C3%B3%C5%BCnice_pomi%C4%99dzy_protoko%C5%82ami_POP3_i_IMAP, Sept. 2021.
- [8] *TELNET - słownik pojęć technicznych*.
https://www.atel.com.pl/slownik_produk_t_ref.php?haslo=TELNET, Sept. 2021.
- [9] *Protokół SNMP (Simple Network Management Protocol)*. <https://www.ibm.com/docs/pl/spectrum-control/5.3.3?topic=standards-simple-network-management-protocol>, Sept. 2021.
- [10] *IRC*. <http://www.irc.pl/>, Sept. 2021.
- [11] *ARP*. https://www.atel.com.pl/slownik_produk_t_ref.php?haslo=ARP, Sept. 2021.
- [12] *8P8C*. <https://pl.wikipedia.org/wiki/8P8C>, Sept. 2021.
- [13] *Reduced Media-Independent Interface*.
https://en.wikipedia.org/wiki/Media-independent_interface, Aug. 2021.
- [14] Rafał Kozik. “Ethernet w FPGA”. In: *Programista* 91.4 (2020), pp. 18–32.
- [15] *ENC28J60 Data Sheet. Stand-Alone Ethernet Controller with SPI Interface*. Sept. 2021.

- [16] *W5500 Datasheet*. Sept. 2021.
- [17] *ESP8266 - co warto wiedzieć ?* <https://forbot.pl/blog/leksykon/esp8266>, Sept. 2021.
- [18] *ESP32 - co warto wiedzieć ?* <https://forbot.pl/blog/leksykon/esp32>, Sept. 2021.
- [19] *Lightweight TCP/IP (lwIP) Stack*. <https://www.analog.com/en/design-center/evaluation-hardware-and-software/software/adswp-lwip.html>, Sept. 2021.
- [20] *Using the Stellaris Ethernet Controller with Micro IP (uIP)*. <https://www.ti.com/lit/an/spma026b/spma026b.pdf?ts=1631990328536>, Sept. 2021.
- [21] *UM1974 User Manual STM32 Nucleo-144 Boards*. Nov. 2021.
- [22] *[STM32 HAL] Ethernet*. Dec. 2021.